
Sales Data Modeling and Changepoint Detection

In collaboration with Maki A/S



Project, 8. semester
Group cs-24-dvml-8-01

Aalborg University
The technical faculty of Computer Science
Selma Lagerløfts Vej 300
DK-9220 Aalborg Øst



STUDENT REPORT

Faculty of Computer Science

Data science

Selma Lagerløfs Vej 300

9220 Aalborg Øst

<http://cs.aau.dk>

Theme:

Time Series Analysis & Changepoint Detection

Project:

P8 - Early Warning System with Bayesian Changepoint Detection

Period:

February 2024 - May 2024

Project Group:

Group cs-24-dvml-8-01

Participants:

Emil Marcus Buchberg
Frederik Hecter Kowalski
Kent Vugs Nielsen

Supervisor:

Sarah Hasan

Page Number: 45 + front page

Project finished: July 6, 2026

Abstract:

Businesses often face common challenges related to time-dependent data. In this study, we address the necessity for an effective early warning system capable of detecting changes in consumer behavior, enabling timely responses such as restocking supplies or canceling orders. This research is particularly focused on Maki's need for such a system. We employ the models: Bayesian Online Changepoint Detection (BOCPD), Pruned Exact Linear Time (PELT), and Prophet. Prophet is utilized to decompose and accurately model the temporal data. Our study involves simulating a real-world scenario for Maki, where the efficacy of BOCPD and PELT in detecting changepoints is evaluated. The results indicate that both BOCPD and PELT can successfully identify changepoints and accurately determine their occurrence. Notably, PELT achieved an F1 score of 0.92 in detecting large abrupt changes, demonstrating its heightened sensitivity to such shifts. In contrast, BOCPD attained an F1 score of 0.75 under the same conditions but exhibited a lower Hamming distance when detecting small continuous changes, indicating its superior performance in identifying more subtle shifts in data.

Preface

The project is written by group cs-24-dvml-8-01 from the master's degree in data science in the 8th semester. The project is written from the beginning of February 2024 to the 29th of May 2024. We leveraged the assistance of ChatGPT [1] and GitHub Copilot [2], which provided valuable code suggestions, expediting the development process and serving as a source of inspiration when encountering challenges. Special thanks to Henrik Rennison Hansen, CFO at Maki A/S, for providing data and guidance, and Sarah Hasan for supervising the project.

Aalborg University, July 6, 2026

Emil Marcus Buchberg
<ebuchb20@student.aau.dk>

Frederik Hecter Kowalski
<fkowal20@student.aau.dk>

Kent Vugs Nielsen
<kniels18@student.aau.dk>

Contents

Preface	i
1 Introduction & related work	1
1.1 Introduction	1
1.2 Related work	2
1.3 Problem statement	3
2 Data analysis	3
2.1 Introduction to data	3
2.2 Descriptive statistics	4
2.3 Data cleaning	6
2.4 Patterns over time	8
2.5 Time series decomposition	11
2.6 Decomposition and data challenges	15
3 Prophet	16
3.1 Trend	16
3.2 Seasonality	16
3.3 Holidays & events	17
3.4 Additional regressors	17
4 Changepoint detection	18
4.1 Bayesian online changepoint detection	18
4.1.1 Recursive run length posterior estimation	19
4.1.2 Changepoint prior	20
4.1.3 Predictive distribution	20
4.1.4 Algorithm	21
4.2 Pruned exact linear time	22
5 Feature engineering and simulation	23
5.1 Time-related feature engineering	24
5.2 Generating test data	27
6 Experiments	29
6.1 Prophet configuration	30
6.2 Changepoint detection experiments	31
6.3 Real world changepoint detection experiment	32
7 Results & discussion	34
7.1 Prophet model hyperparameters	34
7.2 Changepoint detection results	36
7.3 Real world experiment results	38
8 Conclusion and future work	41
9 Bibliography	42
A Appendix - Brand and item overview	45

B Appendix - Optimal Partitioning	45
--	-----------

1 Introduction & related work

1.1 Introduction

This project revolves around time series analysis, an indispensable tool for businesses aiming to gain insight into time-dependent data. This analytical approach involves examining sequences of data points collected at consistent intervals to discern patterns, predict future trends, and make data-driven decisions. Businesses frequently face challenges associated with time-dependent data, particularly in forecasting future conditions. The primary difficulty stems from the inherent nature of time series data, characterized by trends, seasonality, and cyclic variations. Each element requires sophisticated modeling techniques to interpret and predict future behaviors accurately. Common challenges include dealing with missing data, understanding complex seasonal patterns, and distinguishing between short-term fluctuations and long-term trends [3] [4].

Moreover, the assumption that past patterns will predict future trends can be problematic, especially in rapidly changing market conditions or industries. It is, therefore, prudent not to indiscriminately apply all available data. Instead, a careful analysis of the quantity of historical data points necessary for effective model generalization is recommended. Businesses must also navigate the volume and velocity of data, as the increase in real-time data collection demands faster and more efficient analytical techniques. Successfully addressing these challenges enables businesses to optimize operational efficiencies and enhance their competitive edge by proactively anticipating market demands and adjusting to economic shifts. The project is done in collaboration with Maki A/S; therefore, the domain is narrowed down to the company's areas of interest. Maki is recognized as a prominent distributor of toys within the Nordic region, concentrating its commercial activities across Denmark, Sweden, Norway, and Finland. The company operates solely in a business-to-business (B2B) model, representing over 60 brands distributed across over 5000 stores and online retail platforms throughout the Nordic countries [5].

Every month, Maki records all transactions between the company and their clients to analyze historical sales data and conduct forensic evaluations, facilitating more informed decision-making processes regarding future sales strategies. This dataset is also referred to as **sell-in** data. Maki has also provided marketing data for selected **SKU Numbers**. However, the marketing data plays a minor role in this project and thus will not be covered further. In addition, Maki receives data from their customers that details the volume of toys sold directly to end consumers, also referred to as **sell-out** data. One of the key challenges Maki faces is accurately gauging consumer trends by utilizing an early warning system that can detect when consumer trends change in either an upward or downward direction. The analysis of these trend changes, through the application of exogenous variables and historical consumer behavior data, yields insights that empower Maki to make evidence-based decisions regarding inventory management. This analytical process facilitates determining whether to cancel or increase orders contingent upon sales performance. Adopting this approach enables Maki to minimize potential losses and enhance profit realization effectively.

1.2 Related work

Time series analyses are widely utilized across finance, bio-medicine, and business domains. With the wide application field, there exists no single best model or approach to accommodate the varying needs of every sector and every problem [6][7][8][9][10][11]. Thus, the time series research is ever evolving and expanding as requirements and complexity rapidly grow. The classical ARIMA model was proposed in 1970 [12], and to this day, it serves as a solid foundation for time series analysis and predictions. Since then, statistics have been developed and are maintained by national banks and statistical bureaus across America and Europe, such as the X-12-ARIMA and the X-13ARIMA-SEATS and TRAMO implementations, for more sophisticated financial insights and predictions [11][13][14][15]. Several newer approaches are developed with successful results. The interest in deep-learning approaches has spawned many studies that comparably compete with the classical statistical methods [9][16]. Furthermore, foundation models are modified and applied to zero-shot learning and prediction, with promising results [17][18], highlighting the fast-phased evolution of time series analytics.

Maki directly seeks to address trend analysis and leverage the knowledge to guide future business decisions. This problem setting narrows the available models to those that contain or parameterize the trend among other identifiable components in business time series. X-13ARIMA-SEATS and X-12-ARIMA are seasonal adjustment methods developed on the time series decomposition paradigm, perceiving a time series as a construct of 3 components: trend, seasonality, and white noise [11][13][14][15]. Applying such methods can accommodate Maki’s desire for trend analysis. However, the models are developed for financial analysis and accommodate data on either monthly or quarterly data, as opposed to the weekly data provided by Maki. Other models, such as STL decomposition, utilize a decomposition approach without requiring specific temporal spacing[3]. However, to address the specifications of Maki’s proposition, we look toward a model that utilizes a decomposition while parameterizing the components for prediction and analysis. This model is Meta’s Prophet [6], which has shown good performance in business cases [6][19].

To accommodate the warning system Maki has addressed, we combine the modeling of the `sell-out` data, with the research area: changepoint detection. Changepoint detection aims to identify changes in the underlying model or segmental statistical changes in an observed time series [20]. Changepoint detection is leveraged across all domains related to time series and sees a wide range of application approaches [20][21][22]. Changepoint detection has been shown to accommodate financial time series [23] and cryptocurrency trading [24]. An interesting approach that combines the time series decomposition with a changepoint detection framework: “Bayesian Ensemble Algorithm for Change-Point Detection and Time Series Decomposition” (BEAST) developed by Zhao et al. [25]. BEAST utilizes a Bayesian ensemble to decompose and detect changepoints. BEAST returns the components that minimize the model error while simultaneously allowing the trend to change over time. We leverage this philosophy to quantify certainties in trend changes as an early warning system, where the changepoint detection algorithms are directly applied to the Prophet model residuals. Any changepoints in the series should be directly reflected in the residuals and, therefore, be noticed within the 12-week time frame Maki desires.

We chose two changepoint algorithms, Bayesian Online Changepoint Detection (BOCPD) and Pruned Exact Linear Time (PELT). A comparison of changepoint detection algo-

rithms performed by Burg et al. [26] shows that BOCPD and PELT are among the best performers. We aim to leverage this for the purpose of an early warning system.

1.3 Problem statement

In a pursuit to detect market changes at an early stage, increasing Maki's reactivity to trend shifts in the toys market, we model consumer behavior by:

- decomposing the sales data into workable components,
- inferring the underlying trends,
- validating results utilizing statistical test tools.

We detect trend-shift signals in a timely manner by:

- implementing existing changepoint detection algorithms to identify consumer trend changes,
- developing a setup for real-world application.

2 Data analysis

Data analysis aims to get a deeper understanding of the data with descriptive statistics and time series analysis. It will also assist the later feature engineering process by providing informative findings on the variables featured in the data.

2.1 Introduction to data

The dataset employed in this study is sourced from Maki and encompasses sales transactions to Danish consumers. Comprising 11.878 records, each entry in the dataset corresponds to an individual sales transaction. Several attributes, including the sales quantity and the date of sale, with weekly granularity, fabricate these transactions. Additionally, each record includes details of the product involved, specifically the item and brand names. Maki partially anonymizes the dataset, wherein the item names are obscured, and information pertaining to the sellers is omitted.

The temporal data spans from Week 1 of 2022 to Week 10 of 2024. Since the observations contain more than one variable, the time series is multivariate. Additionally, the series is discrete, as the data observation is performed at fixed time intervals with equal spacing.

Table 1: Sales data for Danish consumers.

Brand	SKU Number	Sales	Year	Week	Country
DR	Item B-1	5.0	2022	1	Denmark
DR	Item B-2	4.0	2022	1	Denmark
DR	Item B-3	NaN	2022	1	Denmark
DR	Item B-4	NaN	2022	1	Denmark
DR	Item B-5	17.0	2022	1	Denmark

Table 1 represents the first five rows in the dataset and the header. **Brand** represents the brand name, **SKU Numbers** represents the item name, **Sales** reflects the transactional quantity, **Year** indicates the annum of the transaction, **Week** specifies the week of the year in which the sale occurred, and **Country** represents the country of the seller and where the transaction took place. This exposition of preliminary data provides a foundational understanding for the subsequent section, which engages in a descriptive dataset analysis. The forthcoming analysis synthesizes and interprets these attributes to elucidate underlying patterns, trends, and distributions inherent within the dataset.

2.2 Descriptive statistics

This analysis leverages statistical metrics to provide a detailed quantitative data summary. The exploration includes measures of central tendency such as means and medians, variability indicators like standard deviations, and extremity values delineated by minimum and maximum observations. By examining these metrics across the various data dimensions, we aim to elucidate patterns and tendencies inherent within the sales transactions. The descriptive statistics serve as a lens through which the data's structure and underlying patterns become discernible, guiding further analytical investigation.

Table 2: Descriptive statistics of sales data by brand (14 Brands).

Brand	count	mean	std	min	25%	50%	75%	max
DR	483.0	24.6	38.8	-2.0	3.0	11.0	35.0	320.0
Fantasma	197.0	16.1	30.7	-19.0	4.0	8.0	13.0	195.0
Fortnite	114.0	13.7	21.0	0.0	4.0	6.0	14.75	139.0
Hape	947.0	14.7	22.8	-11.0	4.0	8.0	18.0	201.0
Krea	323.0	9.9	10.8	-2.0	2.0	7.0	13.0	62.0
Magformers	452.0	30.0	53.4	-14.0	4.0	16.0	33.0	488.0
Mattel Games	20.0	410.5	215.7	221.0	276.0	305.5	522.0	986.0
Monster Jam	114.0	104.7	44.8	39.0	77.25	93.0	120.5	304.0
Paw Patrol	342.0	156.0	152.0	23.0	90.0	115.0	157.0	1597.0
Schleich	5018.0	20.5	27.9	-7.0	7.0	13.0	21.0	318.0
Tech Deck	344.0	68.7	104.1	-1.0	10.0	19.5	115.0	699.0
Vildkatten	309.0	78.3	135.9	-24.0	5.0	39.0	100.0	1100.0
Vtech	1711.0	36.7	55.4	-60.0	12.0	21.0	40.0	718.0
Wheely Bug	24.0	1.2	1.0	-1.0	1.0	1.0	2.0	3.0

Table 2 shows salient outcomes from the dataset. Notably, we observe that out of the 14 brands, **Mattel Games** and **Wheely Bug** are represented by 20 and 24 records in the **count** column. These values represent the number of distinct weeks with one or more

transactions. Every brand has a one-to-many relationship with **SKU Numbers**, and every distinct week an item is sold, it is counted under the brand name. Despite this quantitative parity in the dataset's frequency, a contrast is evident in the mean sales quantity, with **Mattel Games** possessing an average of 410.5. This average suggests that the sales transactions associated with **Mattel Games** are relatively substantial in magnitude, albeit infrequent. Conversely, **Wheely Bug** exhibits a mean sales quantity of 1.2, indicating minimal transactional volumes.

The minimum values column, **min**, reveals the presence of negative figures, which signify the occurrence of product returns. This phenomenon denotes instances where the volume of products returned surpasses the volume sold for a specific brand during a particular week.

The **Number of items**, shown in Appendix A, column, which enumerates the number of stock-keeping units, shows that **Schleich** encompasses a more extensive array of unique items sold compared to other brands. This diversity could contribute to the aggregate sales volume of the brand, positing the possibility that a greater diversity of available items may correlate with heightened total sales.

Table 3: Descriptive Statistics of Sales by SKU Number (110 items)

SKU	count	mean	std	min	25%	50%	75%	max
Item B-1	81.0	6.9	9.3	-2.0	2.0	4.0	10.0	55.0
Item B-2	94.0	7.4	9.7	-2.0	2.0	4.0	9.0	53.0
Item B-3	2.0	1.0	0.0	1.0	1.0	1.0	1.0	1.0
Item B-4	1.0	1.0	NaN	1.0	1.0	1.0	1.0	1.0
Item B-5	108.0	30.1	25.7	-1.0	8.0	24.5	46.0	110.0
...	...	...	...	...	...	...	...	...
Item S-9	124.0	41.5	79.1	-34.0	13.0	21.0	30.2	634.0
Item T-1	5.0	1.6	1.6	-1.0	1.0	2.0	3.0	3.0
Item T-2	5.0	1.6	0.8	1.0	1.0	2.0	2.0	3.0
Item T-3	9.0	0.8	1.0	-1.0	1.0	1.0	1.0	3.0
Item T-7	5.0	1.2	0.4	1.0	1.0	1.0	1.0	2.0

Table 3 exhibits descriptive statistics for a select few sales by **SKU Number** and offers an overview of the distribution and variability of sales data for individual items. Examining the table yields several key observations, notably the variability in the mean sales quantities among different items. For instance, **Item B-5** displays a substantially higher average sales volume than most other items, suggesting it is a popular choice among consumers. A mapping between brands and items can be seen in Appendix A.

The existence of extreme values, as evidenced by the **max** column, suggests that certain items experience sporadic surges in sales volume, potentially due to promotional activities

or seasonal demand. Additionally, the standard deviation values reflect the dispersion of sales volumes around the mean. The variation is markedly high for some items like **Item S-9** and points to inconsistent sales patterns and the presence of outliers.

We observe some of the same tendencies in Table 3 as in Table 2, notably the presence of negative values, which intimate occurrences where item returns have surpassed purchases. The minimal figures recorded in the **count** column indicate data paucity for certain items, suggesting they are underrepresented within the dataset. Underrepresentation potentially shows the necessity for data-cleaning processes to ensure robustness in subsequent analyses.

The descriptive analysis conducted thus far illuminates several patterns within the data that warrant attention. Notably, negative values suggest a higher frequency of returns than purchases for certain items, and the low count of transactions for some items indicates potential underrepresentation or data sparsity. Thus, the subsequent section focuses on data cleaning, ensuring the dataset's integrity is upheld, and solidifying the foundation for future analyses.

2.3 Data cleaning

The preceding section presents statistical metrics of the dataset, elucidating certain irregularities that necessitate data cleaning. It is observed that a subset of sales data is characterized by the presence of NaN values. In this context, NaN entries are legitimate and signify instances wherein a product is either unavailable for purchase due to stock depletion or the item is not launched yet. Conversely, entries registering zero sales indicate products that, despite their availability, do not experience any transactional activity. These zeroes are distinguished from NaN values, representing a materially different phenomenon. Additionally, it is stated that negative sales are valid and denote instances where the volume of items returned surpasses the volume sold.

Figure 1 shows the temporal sales trajectories for a select few brands, corresponding to the statistical metrics enumerated in Table 2. Observing the temporal trends for **Mattel Games** and **Wheely Bug** shows the quantitative findings previously reported in Table 2. Specifically, the relative lack of sales transactions. The brands omitted from Figure 1 follow the same trajectory as **Vtech**, **Schleich**, and **DR**. The trend and seasonality are further explained when exploring the data in depth.

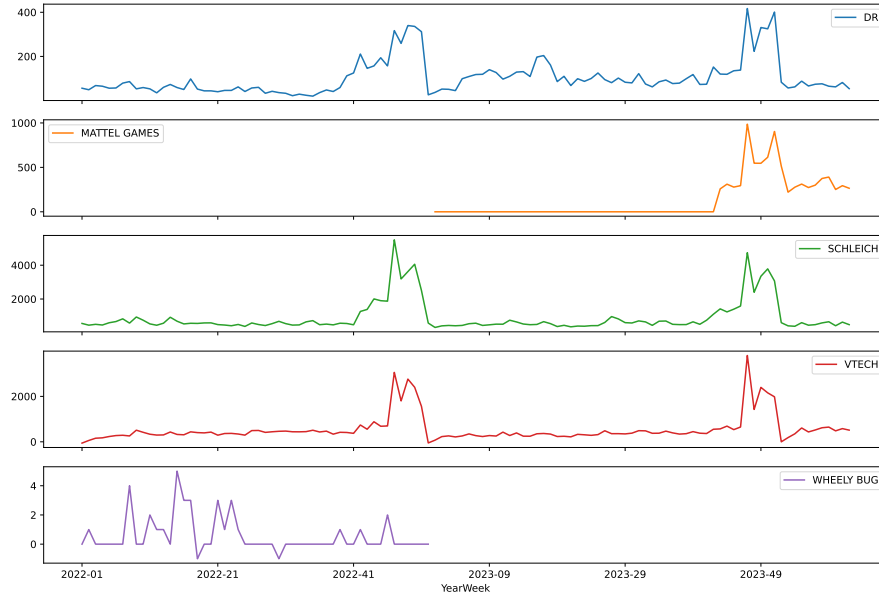


Figure 1: Sales data for select brands

Likewise, Figure 2 shows the temporal sales trajectories for selected items, corresponding to the statistical metrics enumerated in Table 3. Figure 2 shows the infrequent nature of transactional occurrences for specific items within the dataset. The graphical depiction reveals that **Item B-3** registers sales activity solely on two weeks, illustrating a sporadic sales pattern. Similarly, **Item H-4** exhibits a lack of sales, aligning with the aforementioned observation. Further scrutiny exposes **Item H-11** as obsolete, evidenced by its sparse sales records, indicative of its gradual phase-out from active commercial circulation.

The visualization encapsulates a broader trend among a subset of products and brands, including those previously specified, characterized by their sparse representation in the sales data. These items are identified as candidates for exclusion from the dataset in preparation for subsequent analysis. Such removal is justified by the aim of enhancing the dataset's analytical robustness by focusing on items with sufficient data. Thus, items that are neither deprecated nor suffer from significant data sparsity are retained. Examples of such items include **Item K-3** and **Item P-2**, which exhibit a consistent presence of transactional frequency.

After excluding the previously mentioned brands and items, the refined dataset encompasses 12 distinct brands and 94 unique items. This marks the end of the data cleaning phase, setting a foundation for more detailed exploration and analysis.

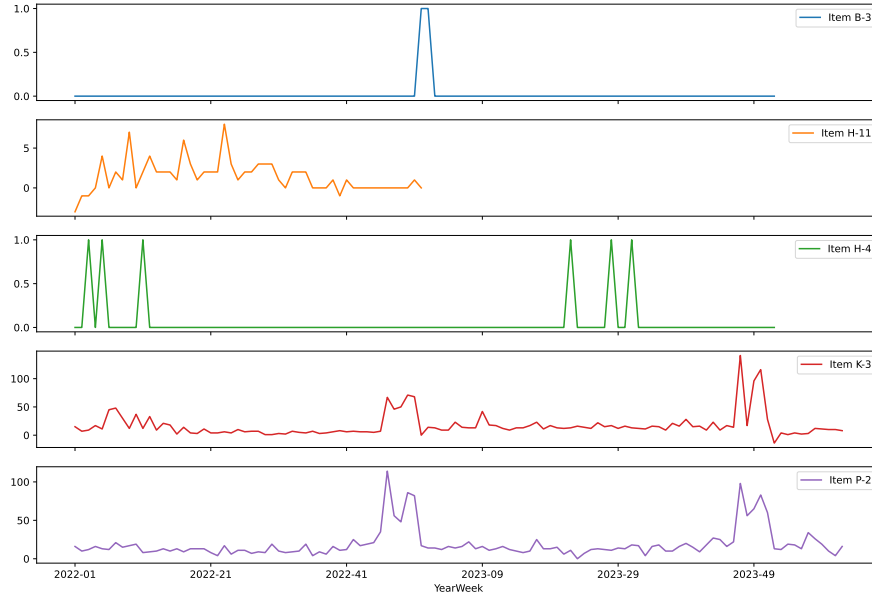


Figure 2: Sales data for select items

2.4 Patterns over time

This section embarks on an explorative time series analysis, focusing on discerning the temporal dynamics of sales across the remaining 12 brands and 94 items. This phase unveils the sequential patterns, trends, and seasonal fluctuations inherent within the sales data. This insight lays the foundation for future modeling. We identify key time-dependent behaviors and anomalies that may influence sales performance by applying domain knowledge and visualizations.

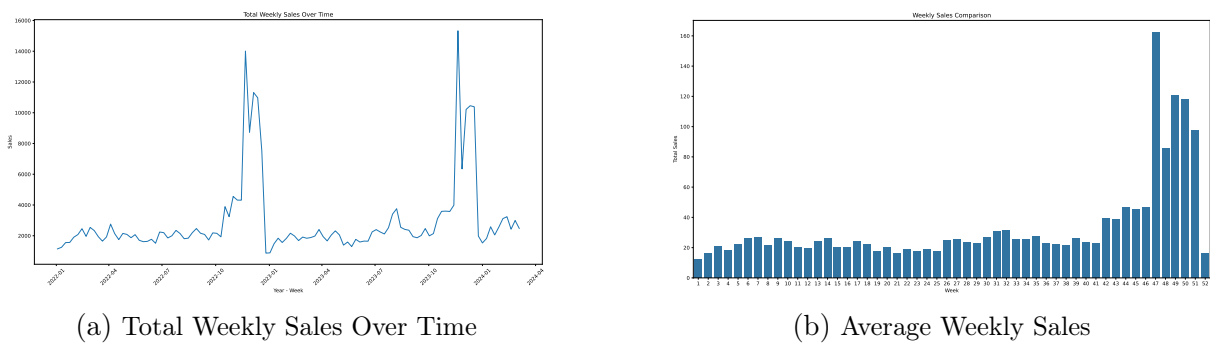


Figure 3: Weekly Sales

Figure 3 provides a quantitative visualization of aggregate weekly sales within the dataset. Figure 3a shows the total sales for each year and week, where Figure 3b depicts the average weekly sales for each week. Figure 3 delineates a relatively stable sales trajectory from the beginning of the calendar year to November. Consistent sales volumes with minor fluctuations characterize this period. Notably, sales escalate considerably in November, culminating in a peak that ends in the last week of the year. The data suggests a seasonal

cycle, with a marked increase in consumer purchasing behavior during this interval, likely attributed to the end-of-year festivities, Christmas sales, marketing campaigns, and the danish black week.

For any time series inference, statistical properties, such as constant mean and variance, are crucial because they are part of stationarity, which is essential for predictive models. We test for the stationarity property by applying the Augmented Dickey-Fuller (ADF) test to the data set. Under the null hypothesis, a unit root is present, and the series is non-stationary. Under the alternative hypothesis, the series is interpreted as mean or trend stationary. We test for both stationary properties and reject the null hypothesis.

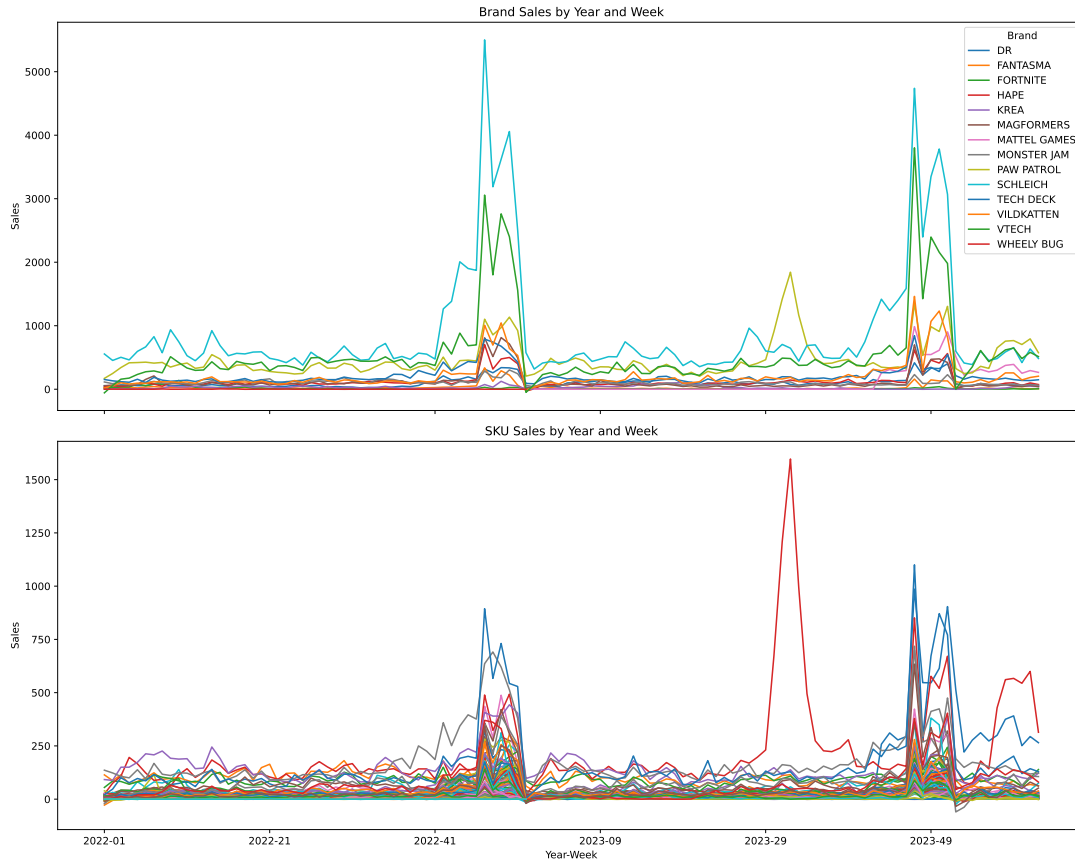


Figure 4: Comparison of Brand and Item Level Sales with x -axis formatted as yyyy-w

Figure 4 visually represents the total sales data, aggregated on brand and item level across temporal segments. The illustration corroborates the general sales trajectory observed in Figure 3a, where most brands and items adhere to the discerned seasonal behavior. Nevertheless, certain brands and items substantially influence the overall sales figures due to their higher sales volumes.

A noteworthy observation from the data is the persistence of sales patterns. Brands registering high sales volumes in one season often have significantly elevated sales in the subsequent season. This suggests consistency in consumer purchasing patterns or possibly the impact of recurring seasonal effects.

Additionally, we identify certain anomalies; specifically, the brand **Paw Patrol** demonstrates an atypical surge in sales between weeks 29 and 36. This deviation from the norm

is largely due to the item **Item 0-2**, underscoring the significance of understanding brand and item performance over time. In the case of brand- and item-level analysis, the binary marketing data supplied by Maki confirms **Paw Patrol** to include marketing campaigns that might influence the sales. However, the binary marketing data only exists for a few brands in the dataset and is consequently not considered in this study. Further analysis is required to classify **Item 0-2** as an outlier. Outliers are often removed from the data, however, in conjunction with the sparse data available, removing outliers will directly harm the model fitting in the later stages. Strategies, such as imputing, are considered if necessary.

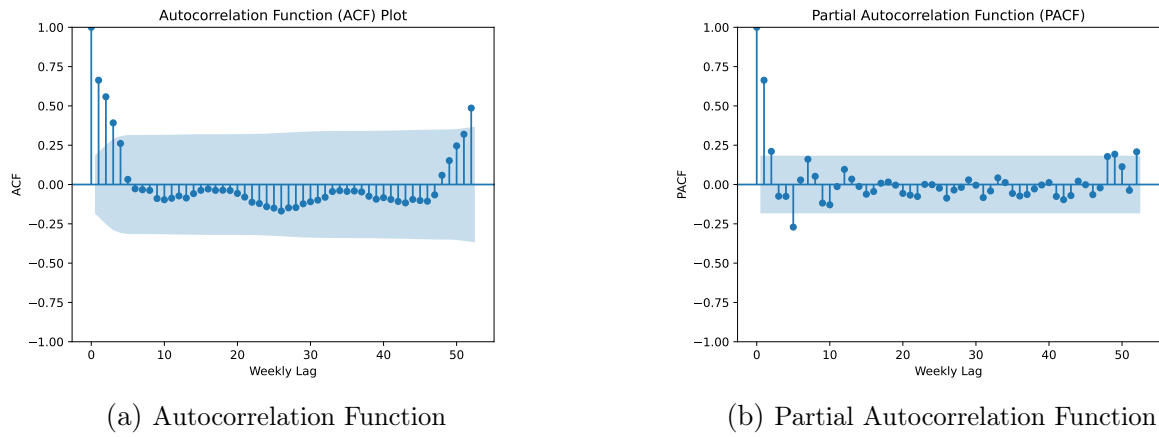


Figure 5: Lagged Sales Correlations

Figure 5a and Figure 5b show the Autocorrelation Function (ACF) and the Partial Autocorrelation Function (PACF), respectively. ACF and PACF are crucial tools in time series analysis for identifying the model type applicable to the data. ACF quantifies the linear relationship between lagged values of a time series, where the PACF isolates the correlation between lagged values based solely on their relationships without confounding influences from other lagged observations. The x -axis delineates the lags on a weekly basis, commencing with lag zero. By definition, lag zero will exhibit a correlation coefficient of one, denoting a perfect correlation, as it represents the degree of association of the sales with itself. After this, the autocorrelation value at lag one quantifies the degree of linear relationship between the sales of the current week and those of the preceding week. This temporal dependency captured at lag one indicates the immediate impact of the prior week's sales on the current week. A set of 52 lags, encompassing lag zero, has been employed to encapsulate an entire annual cycle. The analysis spans the entire spectrum of short-term correlations, which may reveal monthly patterns, as well as long-term correlations that have the potential to uncover underlying annual patterns. The correlation values outside the confidence interval, portrayed as a shaded and bounded area in the figures, are considered statistically significant. A threshold at 95% confidence level defines the area. Since we cannot reject the trend-stationarity, we can assume the significant lags are due to seasonality or other closely related effects. Figure 5 reveals significant monthly and annual behavior. This will be important later during the model selection process.

Subsequent analysis focuses on disentangling the components embedded within the time series. Time series decomposition is a pivotal analytical technique to isolate these elements, providing clarity on the deterministic structure versus stochastic noise. It will offer

us a more granular understanding of the data, elucidating whether the observed patterns are a result of long-term trends, recurrent cycles, or irregular fluctuations.

2.5 Time series decomposition

Identifying and measuring the various effects of time series, akin to the previously identified seasonality, is a rigorous task. This section provides an intuitive insight into comprehensibly decomposing and analyzing time series components.

The seasonal effect on consumer behavior is evident when visually inspecting Figure 3a. It is reasonable to expect sales to increase in the weeks before Christmas and a drop in the last week of the year. Such seasonality is prevalent throughout the series, and one can estimate its effect as a seasonal component. Subtracting the seasonal component exposes the underlying tendencies in the time series. One such tendency is the time-dependent component trend. Trend is coherent with many definitions across domains, but we reason that: “...trend is produced by slowly evolving preferences, technologies, institutions, and demographics” [7, p. 179]. We simultaneously believe that trends are susceptible to abrupt changes, as argued by Zhou et al. [25, p. 2], and shifts in sales behavior should be embedded and reflected by trend changes.

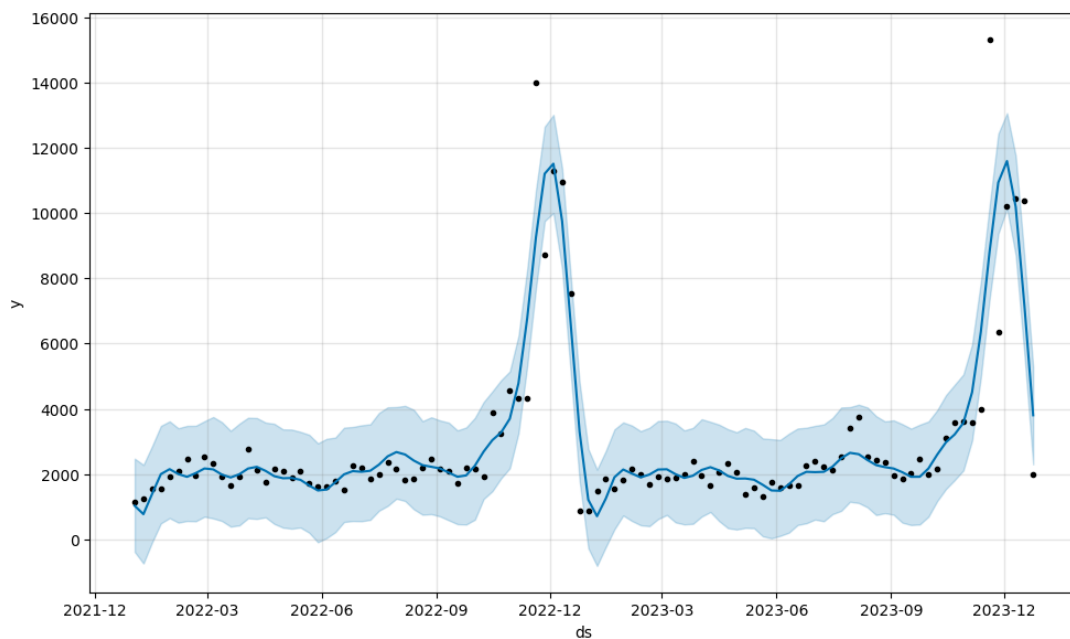


Figure 6: Prophet model on total sales, 2022-2023.

Decomposition using Prophet

In our quest to understand the data, Prophet offers an intuitive and accessible framework. While forecasting models, such as those from the ARIMA family, attempt to make time series stationary and remove parts like trend and seasonality, Prophet estimates these components and their interactions directly. This has the advantage that we can easily analyze and understand the overall structure of the data [3]. Taylor et al. [6] state they are “*framing the forecasting problem as a curve-fitting*” [6, p. 7], which do not explicitly account for the sequential dependence in time series data, offering a flexible and robust

model for both non-stationary and seasonal data. Different algorithms provide various implementations of the decomposition. We use a simplified version of Meta’s Prophet [6], providing a straightforward decomposition. The simple additive time series decomposition is written as

$$y_t = S_t + T_t + \epsilon_t \quad (1)$$

where y_t is data at time t , S_t is the seasonal component, T_t is the trend and ϵ_t is white noise [3]. The white noise term, ϵ , is considered residuals in a model-response. We assume $\epsilon \sim i.i.d. \mathcal{N}(0, \sigma^2)$, that is, the residuals are normal, independent, and identically distributed (i.i.d.), around 0 mean, with some variance σ^2 .

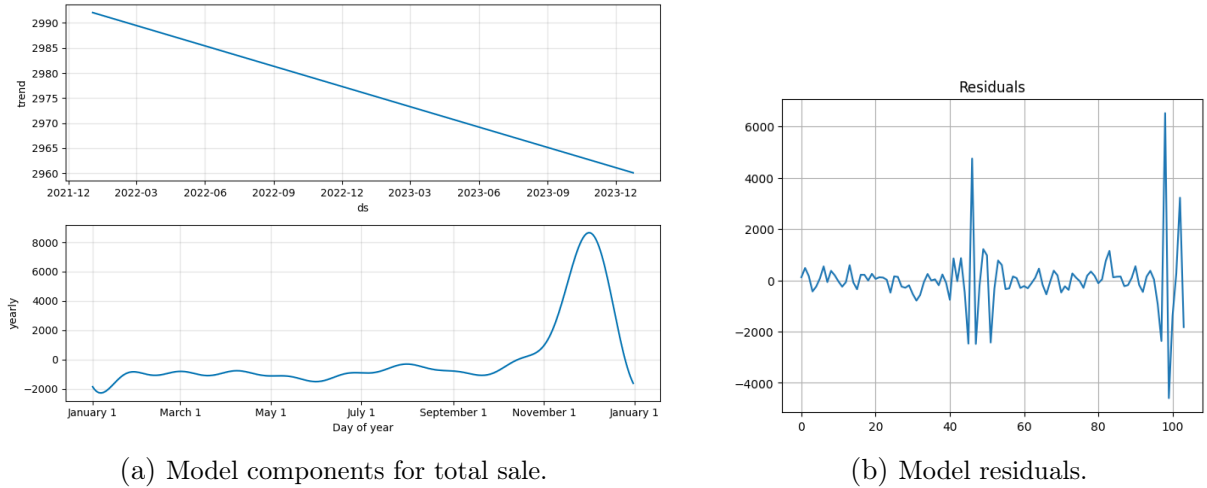


Figure 7: Prophet decomposition and residuals of total sales

We treat 2024 as a test set and conduct our decomposition analysis in the first two years. Excluding the ten weeks in 2024, the model shown in Figure 6 is fit on a granularity of total sales. The model appears to capture the general behavior of the time series appropriately, and the estimated components are visualized Figure 7a. The model is sanity-checked by plotting the residuals observed on Figure 7. However, residuals resulting from a successful fit exhibit characteristics of Gaussian i.i.d. and provide a clear interpretable method to validate the fit.

By visual inspection of Figure 7b, the residuals are neither normal nor i.i.d. The mean appears centered around 0, but the variance is not constant. Testing the residuals with an ADF test yields a p-value of 0.0003, and the series can be considered trend-stationary. A Shapiro-Wilk test, applied for testing normality, confirms the residuals are not normally distributed. However, the trend-stationarity and mean of the residual series, valued at 0.09, suggest a foundation for future modeling.

The weeks, 47 – 50, exhibit the largest variational residual behavior. Given the extreme peaks observed in Figure 3, compared to the captured seasonality of the same period in Figure 7a, the residuals’ behavior is expected. Instead of attempting to model the extreme changes directly with the seasonality component and potentially overfit the yearly seasonality, the Prophet framework allows further modeling of holiday effects, enabling the encapsulation of calendar events. We expand Equation (1) with a holiday term, to obtain

$$y_t = S_t + T_t + H_t + \epsilon_t, \quad (2)$$

and include the weeks 47 – 50 as holidays.

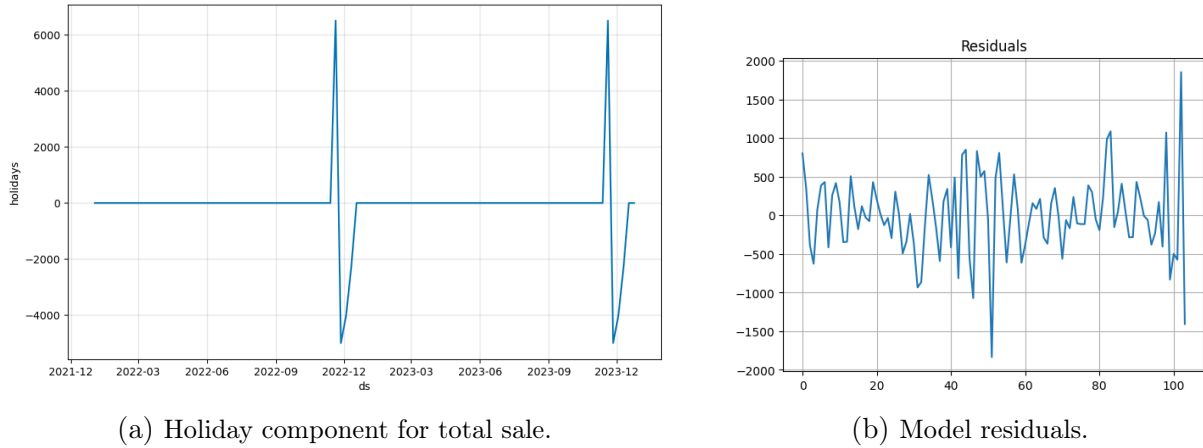


Figure 8: Prophet decomposition with holidays and residuals of total sales.

Figure 8a shows the validity of adding holidays. The holiday component models the sudden spikes in December, which the default seasonality component cannot reasonably capture. The residuals appear less volatile, shown on Figure 8. However, the Shapiro-Wilk test determines the residuals are not yet normally distributed. Again, we see a substantial variance in the residuals during weeks 47 – 50. We try to reduce this variance with a multiplicative model. Equation (2) is written on the form

$$y_t = S_t \cdot T_t \cdot H_t \cdot \epsilon_t. \quad (3)$$

A multiplicative model achieves slightly less residual variance, as observed on Figure 9a. The residuals' normality cannot be rejected with a Shapiro-Wilk test and visually appears normal when plotted in a quantile-quantile plot, Figure 9b. The residual pattern resembles the additive model's and does not appear to be i.i.d. The residual behavior is not entirely addressed, but Meta's Prophet provides a promising model for further modeling and analysis.

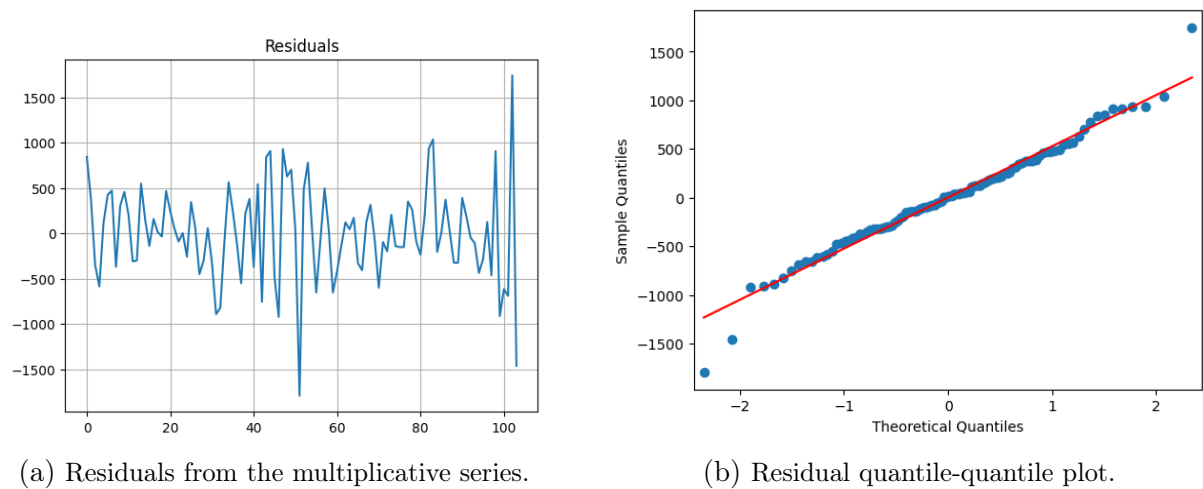


Figure 9: Prophet decomposition with holidays and residuals of total sales.

Piece-wise linear trend

The trend present in Equation (3) is constant linear. It is necessary to cast the linear trend assumption in a different construct to accommodate the presupposition of trend changes. If an identifiable mean shift of the series has appeared, it is appropriate to model the trend with the inherit information in a segmented manner. That is, to identify and estimate significant slope changes in the trend. Prophet employs trend segmentation by applying an evenly spaced grid on the time series, where the magnitude of change is evaluated. These points are changepoints.

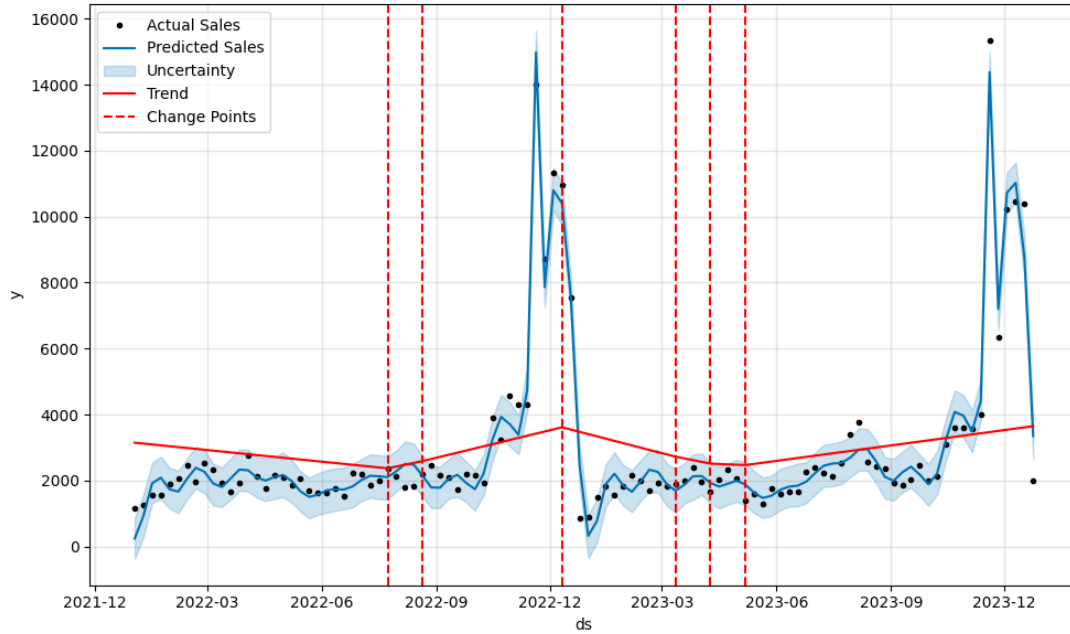


Figure 10: Prophet model with 25 changepoints on total sales, 2022-2023. 6 changepoints are significant and plotted.

Utilizing changepoints permits a flexible trend. Prophets approaches changepoints by applying regularization to each changepoint, relying on evidence from the data to adjust trend flexibility. Relaxing the regularization permits more significant slope changes on evaluated changepoints.

Figure 10 shows how changepoints can capture segments of shifting behavior. The model is fit with 25 changepoints, of which 6 are significant. The magnitudes can be observed on Figure 11. Magnitudes above 0.01 and below -0.01 are considered significant with Prophet's default settings and are thus plotted on Figure 10. Prophet uses the first 80% of the series to place the evenly spaced changepoints. The default regularization is relaxed to encourage a more

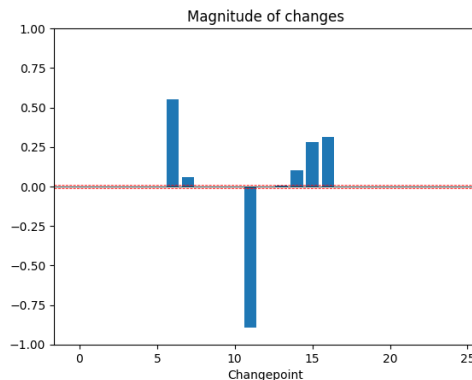


Figure 11: Change point magnitudes where each bar represents relative change compared to the previous changepoint.

flexible trend, allowing the model to capture trend changes. With no exogenous variables, a flexible trend comes with a trade-off. The trend can model missing exogenous information; however, additionally, relaxing regularization allows the trend to creep explainability from other components. Observing the trend slopes of Figure 10 incites suspicion of seasonality. The trend contains a downward trajectory through the first six months and shifts upwards, closing in on Christmas, resembling a yearly occurrence. Fitting a model with a flexible trend may reduce residual variability. However, it poses an overfitting obstacle.

2.6 Decomposition and data challenges

Section 2.5 introduces several workable components for examining and reasoning with trend behavior. The approach we adopt from Prophet is a curve-fitting exercise. Unlike traditional methods such as ARIMA, Prophet allows fewer constraints when fitting the model [6]. The challenge then becomes identifying, adding, and adjusting components that best describe the behavior of the time series. Section 2.5 can be understood as a step-by-step approach to gradually fit a model and identify potential obstacles. Fitting the model on Figure 10 yields normal distributed residuals.

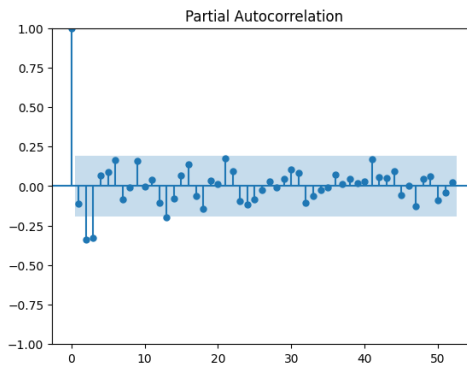


Figure 12: PACF for Figure 10

Plotting the PACF for the residuals, observed on Figure 12, evidently shows the residuals are not yet i.i.d., as some lags are significant. The residuals require further examination combined with feature engineering to overcome the sequential correlation. This problem becomes complicated as more variables and feature engineering rapidly introduce model complexity and overfitting potential.

The data consists of two complete seasonal cycles and provides a challenging decomposition. The seasonality can be pinpointed visually, but estimating its effect with a few cycles is difficult. Figure 10 highlights identical seasonal patterns, but the yearly difference around November and December makes it complicated to estimate what component effects can be attributed to the decomposition. Seasonality estimation is a model specification, which makes its evaluation susceptible to model selection biases. The potential of utilizing 2024 data as a test set for changepoint algorithms leaves the model parameter estimation and evaluation entirely to 2022-23 when avoiding look-ahead bias.

Standard methods such as time series cross-validation exclude the last training points in 2023 for evaluation and, therefore, do not fully utilize the two complete seasonality cycles. If we rely only on cross-validation of the training set, which is the complete 2022-2023 data, the end of 2023 will be underestimated by the seasonal factor, observed on Figure 10. Hyperparameters that allow hyper-flexible trends can model the lack of seasonal cycles and exogenous explainable variables and are favored in model selection. Possibly more sophisticated methods can be considered for model selection. Choosing the best available hyperparameters for our decomposition model directly impacts the performance of future changepoint detection algorithms.

3 Prophet

We introduce two specifications of Prophet in Section 2.5. The additive model is specified in Equation (2) and the multiplicative model in Equation (3). We aim to delineate how the individual components of the model are estimated and introduce extensions to the established model specification.

3.1 Trend

Taylor et al. [6] propose two trend models: a saturating growth model and a piecewise linear model. The former implements a logistic growth function constrained by a user-defined upper limit, known as the carrying capacity. This capacity represents the maximum achievable value for the response variable. Since our data encompasses a selected subset of products in an unknown number of stores, defining a precise carrying capacity is infeasible. Therefore, we use and present the latter.

The piecewise linear trend model adjusts its trend over time. An adjustable growth rate has the benefit that it can handle evolving trends and capture shifts in behavior that we cannot attribute to the other components in the model. We refer to the points in time where the growth rate changes as changepoints. Suppose we have S changepoints at times $s_j, j = 1, \dots, S$ where $s_1 < s_2 < \dots < s_S$. The growth rate at a time-step t is then found by adding the base rate k with all the adjustments up to that point: $k + \sum_{j:t > s_j} \delta_j$, where $\boldsymbol{\delta} \in \mathbb{R}^S$ is a vector of rate adjustments. As a shorthand, this can be represented as a vector $\mathbf{a}(t) \in \{0, 1\}^S$ by rule

$$a_j(t) = \begin{cases} 1 & \text{if } t \geq s_j, \\ 0 & \text{otherwise.} \end{cases}$$

The rate at time t can then be expressed as $k + \mathbf{a}(t)^T \boldsymbol{\delta}$. For the forecasted trend to remain continuous despite rate changes, the offset parameter m is adjusted to connect the endpoints of the piecewise segments. The adjustment at changepoint j is computed as

$$\gamma_j = \left(s_j - m - \sum_{l < j} \gamma_l \right) \left(1 - \frac{k + \sum_{l < j} \delta_l}{k + \sum_{l \leq j} \delta_l} \right).$$

Collecting the pieces, we can now define the trend as

$$g(t) = (k + \mathbf{a}(t)^T \boldsymbol{\delta})t + (m + \mathbf{a}(t)^T \boldsymbol{\gamma}).$$

Detection of changepoints is done automatically by placing a sparse prior on δ_j . Taylor et al. [6] suggest to use the Laplace distribution; $\delta_j \sim \text{Laplace}(0, \tau)$, where τ controls the scale of the distribution. Thus selecting increasingly larger values of τ increases the model's flexibility when adjusting the growth rate. On the contrary, setting $\tau = 0$ reduces the trend to (non-piecewise) linear growth.

3.2 Seasonality

The seasonal component quantifies recurring variations caused by the response variable's time dependency. Prophet can model daily, weekly, monthly, and yearly dependencies or

any combination thereof. Prophet does this with a Fourier series, meaning a sum of sine and cosine functions

$$s(t) = \sum_{n=1}^N \left(a_n \cos \left(\frac{2\pi nt}{P} \right) + b_n \sin \left(\frac{2\pi nt}{P} \right) \right),$$

where P represents the period of the seasonality (e.g. $P = 7$ for weekly seasonality), N represents the number of Fourier terms, and a_n and b_n represent the Fourier coefficients. Increasing N promotes the modeling of faster changes in the seasonal patterns but also makes it more prone to overfitting. Fitting seasonality involves estimating $2N$ parameters: $\beta = [a_1, b_1, \dots, a_N, b_N]^T$. Additionally, in a forecasting scenario, a normal prior, $\beta \sim \text{Normal}(0, \sigma^2)$, is used to impose a smoothing effect on the Fourier coefficients.

The choice of N depends on the data's period and nature. However, Taylor et al. [6] found a Fourier order of 10 works well for yearly seasonality.

3.3 Holidays & events

The holiday component captures the foreseeable but often non-periodic patterns. For example, in the Maki dataset, a significant shift in sales occurs right before Christmas, caused by various consumer holidays.

By assuming that the holidays' effects are independent, Taylor et al. [6] propose a straightforward method to quantify their effect. Each holiday has its own coefficient and is treated as an independent variable. If we let D_i be the set of dates of a holiday i , we can represent this in a compact form

$$Z(t) = [\mathbf{1}(t \in D_1), \dots, \mathbf{1}(t \in D_L)]$$

and assign each holiday to a coefficient κ_i

$$h(t) = Z(t)\kappa.$$

As with seasonality, a normal prior $\kappa \sim \text{Normal}(0, \nu^2)$ is imposed, where ν^2 denotes the variance. In practice, one can define a lower- and upper window around a holiday if it is expected to have an effect outside the holiday itself. The underlying details are no different from regular holidays as they are included in D_i and assigned to a parameter κ_i .

3.4 Additional regressors

As mentioned in Section 2.5, Prophet can easily be extended with additional regressors. The model specification then takes the following form in the additive case

$$y(t) = g(t) + s(t) + h(t) + \sum_i^R \beta_i x_i(t) + \epsilon_t,$$

where $x_i(t)$ are the values of the regressor x_i at time t and β_i are coefficients of the regressors. We can, of course, define an analogous multiplicative model. This straightforward extension of the model is useful in cases where regressors are expected to carry additional information on the response variable.

4 Changepoint detection

Changepoint detection is the task of identifying changes in the underlying model of a signal or time series. This process is essential for understanding shifts in data patterns, which can indicate significant events or transitions. Formally, we frame changepoint detection as a model selection problem. The objective is to choose the best possible data segmentation using a quantitative criterion that must be minimized. This involves identifying the points in the time series where the statistical properties of the data change, thus partitioning the series into segments that are homogeneous within themselves but different from each other.

There are two main branches of changepoint detection methods: online and offline. Online methods aim to detect changes as they occur in real-time, enabling immediate response to events or anomalies. On the other hand, offline methods retrospectively detect changes after all samples have been collected, often referred to as signal segmentation. In this project, we implement an online and offline method to compare their performances in addressing the problem presented by Maki. Maki receives new customer data monthly, necessitating a continuous evaluation of consumer signals. This real-time assessment is important for the timely identification of changes in consumption patterns, enabling Maki to respond effectively to evolving customer behaviors. In the following, we present two methods that aim to solve the problem of detecting an unknown number of changepoints in real-time.

4.1 Bayesian online changepoint detection

The BOCPD algorithm is developed by Adam and MacKay [27]. The following theory is drawn from their paper. Furthermore, the derivations closely follow those of the blog post by Gundersen [28].

BOCPD is an online method that identifies changepoints in data sequences. Formally, a sequence of observations $x_1, x_2, \dots, x_T$ is assumed to be partitioned into non-overlapping segments ρ . The borders between these segments are the changepoints. Data within each segment is assumed to be i.i.d. from a probability distribution $p(x_t | \eta_\rho)$, where a changepoint is a result of a significant change in the distribution's parameters η .

We denote the number of steps since the last changepoint at time t as the run length r_t . The set of observations associated with the run length $r_t = \ell$ are defined as $\mathbf{x}_t^{(\ell)}$. For example, if $r_t = 3$, indicating a changepoint occurred at $t - 3$, the associated observations are $\mathbf{x}_{(t-3):t}$. If $r = 0$, this set is empty.

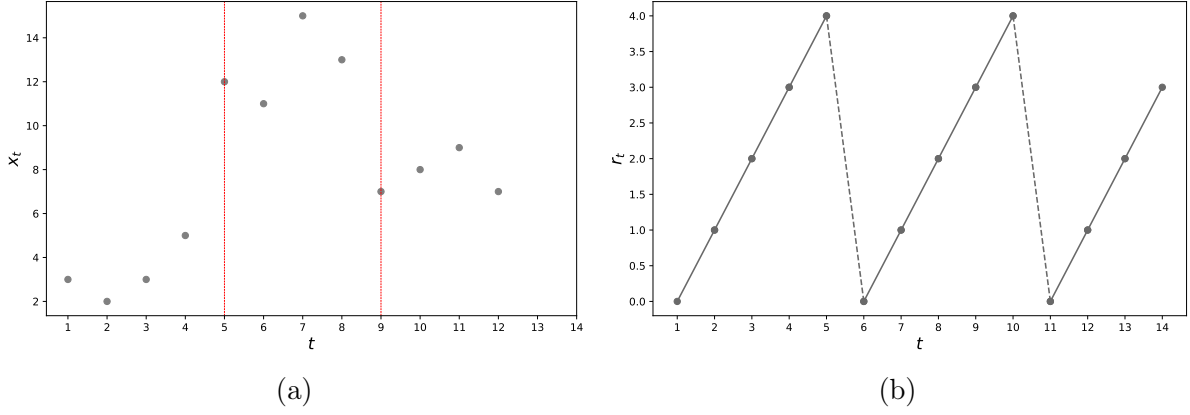


Figure 13: Figure 13a shows generated data with changepoints at $t = 5$ and $t = 9$, resulting in three segments. Figure 13b shows the corresponding run lengths. r_t drops to 0 as a changepoint is detected.

Figure 13b illustrates the relationship between run length and time. As observed, $r_t = 0$ if t is a changepoint; otherwise, the run length increments by 1 ($r_{t-1} + 1$). These are referred to as transition probabilities, and we later describe how we can model any prior knowledge of them with a changepoint prior $p(r_t | r_{t-1})$.

BOCPD then finds changepoints by processing data sequentially and computing a run length probability distribution over the time steps since the last changepoint, including the changepoint probability at the current time step. Given the observed data so far, the posterior distribution over the current run length reveals this information.

When a new data point in the sequence arrives, the predictive distribution $p(x_{t+1} | \mathbf{x}_{1:t})$ is evaluated for all possible run lengths, providing a likelihood measure of the new data for each run length. To get the posterior distribution of r_t , we combine the posterior distribution over r_{t-1} with the likelihood of the new data. This process gives us exactly the probability we are interested in: the probability of a changepoint at time t . The following sections will describe how these distributions are inferred and how they integrate into the overall algorithm.

4.1.1 Recursive run length posterior estimation

At the core of BOCPD lies message passing, a recursive updating procedure of the posterior distribution of run lengths. Formally, the posterior distribution is given as

$$p(r_t | \mathbf{x}_{1:t}) = \frac{p(r_t, \mathbf{x}_{1:t})}{p(\mathbf{x}_{1:t})}.$$

The marginal likelihood $p(x_{1:t})$ is a normalizing constant. We use the law of total probability over the run lengths at $t - 1$ to expand the joint probability $p(r_t, \mathbf{x}_{1:t})$ of the run lengths and observed data:

$$\begin{aligned} p(r_t, \mathbf{x}_{1:t}) &= \sum_{r_{t-1}} p(r_t, r_{t-1}, x_t, \mathbf{x}_{1:t-1}) \\ &= \sum_{r_{t-1}} p(r_t, x_t | r_{t-1}, \mathbf{x}_{1:t-1}) p(r_{t-1}, \mathbf{x}_{1:t-1}) \\ &= \sum_{r_{t-1}} p(x_t | r_t, r_{t-1}, \mathbf{x}_{1:t-1}^\ell) p(r_t | r_{t-1}, \mathbf{x}_{1:t-1}) p(r_{t-1}, \mathbf{x}_{1:t-1}) \end{aligned}$$

The second step applies the chain rule to decompose the joint probability further. In the third step, we make two assumptions: the run length transition probability is independent of past data

$$p(r_t \mid r_{t-1}, \mathbf{x}_{1:t-1}) = p(r_t \mid r_{t-1}),$$

and

$$p(\mathbf{x}_t \mid r_t, r_{t-1}, \mathbf{x}_{1:t-1}) = p(x_t \mid r_t, \mathbf{x}_t^\ell),$$

implying that the likelihood of current observations depends only on the current run length and the past $\mathbf{x}^{(\ell)}$ observations within the current segment. Thereby, we can express the run length posterior as

$$p(r_t, \mathbf{x}_{1:t}) = \sum_{r_{t-1}} \underbrace{p(x_t \mid r_t, \mathbf{x}_t^\ell)}_{\text{Predictive dist.}} \underbrace{p(r_t \mid r_{t-1})}_{\text{Changepoint Prior}} \underbrace{p(r_{t-1}, \mathbf{x}_{1:t-1})}_{\text{Message}}. \quad (4)$$

The recursive nature of computing the run length should now be obvious. The *message*, being the joint distribution $p(r_{t-1}, \mathbf{x}_{1:t-1})$, is propagated to the calculation of $p(r_t, \mathbf{x}_{1:t})$. We are now left with computing the changepoint prior and the predictive distribution over new observations before we obtain the joint distribution over the current run length and the data.

4.1.2 Changepoint prior

A necessary step in the computation of the *changepoint prior* in Equation (4) is a formalization of the idea shown in Figure 13b. At a given time step, the run length either increases by one or is truncated back to zero

$$P(r_t \mid r_{t-1}) = \begin{cases} 1 - H(r_{t-1} + 1) & \text{if } r_t = r_{t-1} + 1 \\ H(r_{t-1} + 1) & \text{if } r_t = 0 \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where $H(\tau)$ denotes the hazard function. It describes how likely a changepoint is to occur at run length τ , given a changepoint has not occurred by τ . A typical choice is to make the hazard function memoryless by selecting a discrete geometric distribution over the current run length, such that $H(\tau) = 1/\lambda$. An alternative is to penalize longer run lengths by making $H(\tau)$ dependent on time. Equation (5) is advantageous in terms of computational efficiency for two reasons: 1) We only assign probability mass at two outcomes. 2) The hazard function needs only to be computed once to compute both cases.

4.1.3 Predictive distribution

The remaining component in Equation (4) is the *predictive distribution*. The authors propose a conjugate-exponential model to compute this distribution as it is particularly convenient in the BOCPD scheme.

Conditioning the predictive distribution is conditional on the run length, and it naturally follows that we can condition on the appertaining data for that run length, meaning all recent data $\mathbf{x}^{(\ell)}$ comes from the same segment. We illustrate this relationship as

$$p(x_t \mid r_{t-1}, \mathbf{x}_{1:t-1}) = p(x_t \mid \mathbf{x}_{t-1}^{(\ell)}). \quad (6)$$

Evaluating the predictive distribution is done with standard Bayesian updating:

$$p(x_t | r_{t-1}, \mathbf{x}_{1:t-1}) = \int_{\eta_\rho} p(x_t | \eta_\rho) p(\eta_\rho | \mathbf{x}_{t-1}^{(\ell)}) d\eta_\rho$$

and

$$P(\eta_\rho | \mathbf{x}_{t-1}^{(\ell)}) = \frac{p(\eta_\rho) p(\mathbf{x}_{t-1}^{(\ell)} | \eta_\rho)}{p(\mathbf{x}_{t-1}^{(\ell)})}$$

Recall that η_ρ denotes the data's parameters within segment ρ . Computing this posterior may be intractable for prior-likelihood combinations that are not conjugate. Using a conjugate model simplifies the computation of the posterior predictive significantly, as it only requires keeping track of hyperparameters of the prior over η_ρ at $t - 1$ [29]. Let ν and χ denote the sufficient statistics of the prior $p(\eta_\rho)$, then a single updating iteration is given as

$$\begin{aligned}\nu_t^{(\ell)} &= \nu_{t-1}^{(\ell)} + v(n) \\ \chi_t^{(\ell)} &= \chi_{t-1}^{(\ell)} + u(x_t),\end{aligned}$$

where u and v are simple functions. Note, at initialization, each parameter value is

$$\begin{aligned}\nu_t^{(0)} &= v_{\text{prior}} \\ \chi_t^{(0)} &= \chi_{\text{prior}}.\end{aligned}$$

Thus, we can simplify and calculate Equation (6) with

$$p(x_t | r_{t-1}, \mathbf{x}_{1:t-1}) = p(x_t | \nu_{t-1}^{(\ell)}, \chi_{t-1}^{(\ell)}).$$

4.1.4 Algorithm

Assembling all the pieces, we can now present the algorithm as its whole. For a recursive algorithm like the one in Equation (4), we must specify its initial conditions. A simple but common scenario is that a changepoint occurred right before the first observation; therefore, we set $p(r_0 = 0) = 1$.

1. Initialize

$$\begin{aligned}p(r_0 = 0) &= 1 \\ \nu_1^{(0)} &= \nu_{\text{prior}} \\ \chi_1^{(0)} &= \chi_{\text{prior}} \\ t &= 1\end{aligned}$$

2. Receive new observation x_t

3. Compute predictive probability.

This is done for each possible run length at time $t - 1$. There are t probabilities $\pi_{t-1}^{(\ell)}$.

$$\pi_{t-1}^{(\ell)} = p(x_t | \nu_{t-1}^{(\ell)}, \chi_{t-1}^{(\ell)})$$

4. Calculate growth probabilities.

The computation is done for each possible run length value $r_t = \ell$:

$$p(r_t = \ell, x_{1:t}) = p(r_{t-1}, x_{1:t-1}) \pi_{t-1}^{(\ell)} (1 - H(r_{t-1}))$$

5. **Calculate changepoint probability.** The calculation is the same as in the previous step, but for the specific case, $r_t = 0$. Note that there is a sum over r_{t-1} because it can be any value between 0 and t .

$$p(r_t = 0, x_{1:t}) = \sum_{r_{t-1}} p(r_{t-1}, x_{1:t-1}) \pi_{t-1}^{(\ell)} H(r_{t-1})$$

6. **Calculate evidence.** The marginal likelihood in Equation (4) that acts as a normalization constant.

$$p(x_{1:t}) = \sum_{r_t} p(r_t, x_{1:t})$$

7. **Compute run length posterior**

$$p(r_t | x_{1:t}) = p(r_t, x_{1:t}) / p(x_{1:t})$$

8. **Update Sufficient Statistics**

$$\begin{aligned} \nu_t^{(0)} &= \nu_{\text{prior}} \\ \chi_t^{(0)} &= \chi_{\text{prior}} \\ \nu_t^{(\ell)} &= \nu_{t-1}^{(\ell)} + 1 \\ \chi_t^{(\ell)} &= \chi_{t-1}^{(\ell)} + u(x_t) \end{aligned}$$

9. **Set $t = t + 1$ and return to step 2**

4.2 Pruned exact linear time

The PELT method, by Killick et al. [30], is an efficient technique for detecting change-points in data sequences. Like the BOCPD method, PELT identifies change-points sequentially. Although PELT shares some of BOCPD's notation, we create a distinction between the two in order to minimize confusion. Thus, we formally denote an ordered sequence of data, $y_{1:n} = (y_1, \dots, y_n)$ and denote the potential changepoint locations as $\tau_{1:m} = (\tau_1, \dots, \tau_m)$, where m is the number of changepoints in the model. Each changepoint position is an integer between 1 and $n - 1$ inclusive. Additionally, we define $\tau_0 = 0$ and $\tau_{m+1} = n$ and assume that the changepoints are ordered. Consequently, the m changepoints will split the data into $m + 1$ segments.

For notational simplicity, it is assumed that the minimum segment length is a single observation $\tau_{i-1} - \tau_i \geq 1$. PELT employs an approach to changepoint detection by minimizing a cost function over all potential numbers and locations of changepoints.

One commonly used approach to identify multiple changepoints is to minimize:

$$\sum_{i=1}^{m+1} [\mathcal{C}(y_{(\tau_{i-1}+1):\tau_i})] + \beta f(m), \quad (7)$$

where $\mathcal{C}$ is a cost function for a segment and $\beta f(m)$ is a penalty to prevent overfitting. PELT is designed for linear cost functions, such as Akaike's Information Criterion (AIC)

that assess the goodness of fit of statistical models while penalizing for complexity to avoid overfitting.

A salient feature of PELT is its ability to guarantee the identification of a global minimum of the cost function. The PELT method is particularly adept at determining the optimal number and locations of changepoints with a computational complexity of $\mathcal{O}(n)$ [30]. This characteristic renders PELT especially suitable for application to large datasets.

One of the primary advantages of PELT is its computational efficiency. It can be orders of magnitude faster than alternative changepoint detection methods such as Optimal Partitioning and Binary Segmentation, while not compromising accuracy [30]. The PELT method includes a critical pruning step, which reduces computational cost without affecting the exactness of the resulting segmentation. In this context, pruning involves the elimination of values of τ that cannot be minima from the minimization process at each iteration. Assuming that when introducing a changepoint into a sequence of observations the cost, $\mathcal{C}$, of the sequence reduces. Formally the pruning rule is given, by assuming there exists a constant K , for all $t < s < T$,

$$\mathcal{C}(y_{(t+1):s}) + \mathcal{C}(y_{(s+1):T}) + K \leq \mathcal{C}(y_{(t+1):T}). \quad (8)$$

then if

$$F(t) + \mathcal{C}(y_{(t+1):s}) + K \geq F(s) \quad (9)$$

holds at a future time $T > s$, then t can never be the optimal last changepoint prior to T and t is thus pruned. $F(t)$ and $F(s)$ denote the minimization of Equation (7) and is further formally defined in Appendix B.

The intuition behind this result is that if the condition in Equation (9) is true, then for any timepoint T later than s , the optimal way to segment the data, with the last changepoint at s , will always be better than having the last changepoint at t . It is an important result because it helps us ignore changepoint candidates t that are unnecessary for finding the final set of changepoints and reduces unnecessary calculations.

5 Feature engineering and simulation

This chapter addresses some fitting and feature engineering challenges presented in Section 2.6. We showcase some of these issues and highlight the problems and potential solutions.

Fitting a Prophet model on the short time frame in which the data has records highlights the difficulties of estimating and attributing the correct effects of components. Figure 9a contains artifacts from December, evidence of the data complexity. We assume the effects around Christmas are related to the amount of days the stores stay open. However, strictly testing this hypothesis is difficult, as two obstructions hinder such analysis. Firstly, having only two cycles makes it cumbersome to attribute concise effects; we cannot isolate the cause or discern a yearly effect from another. Secondly, the data is on a weekly granularity; thus, information is lost from daily granularity, which is beneficial for modeling additional features. Further considerations and time-related feature engineering are necessary.

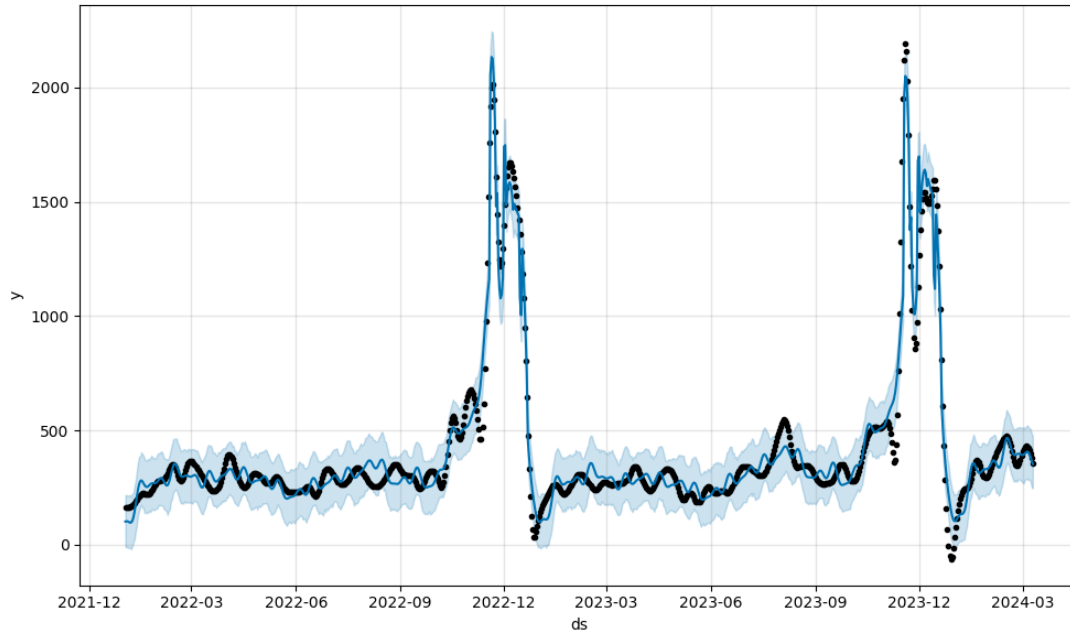


Figure 14: Polynomial interpolated data with a Prophet model fit.

5.1 Time-related feature engineering

Several approaches can potentially improve our model's fit around the troublesome weeks. We consider the granularity as a potential solution, and attempt to solve that first. To obtain data on a lower granularity, we up-sample the dates between our data points to get dates on a daily basis. We then interpolate the values between observed data points. We can perform linear interpolation by letting D_j be the sale on day j in the i th week W_i , the daily interpolated sale is estimated as $D_{i,j} = W_i + (j - 1) \cdot \frac{W_{i+1} - W_i}{7}$, with a rate of change $\Delta D_i = \frac{W_{i+1} - W_i}{7}$. The `pandas` library offers these up-sampling and interpolation methods. We apply linear interpolation, spline interpolation, and second-order polynomial interpolation, observed on Figure 14. A Prophet model is then fit on the new data, having daily granularity. It is, however, evident that the interpolation exhibits the same if not worse, traits than the weekly series. The second-order polynomial interpolation residuals are observed on Figure 15. The residual variability around the problematic weeks appears to increase with interpolation. Thus, it does not appear to provide any new foundation for modeling or solution, and the idea is scrapped. Instead, we attempt to model the calendar effect directly.

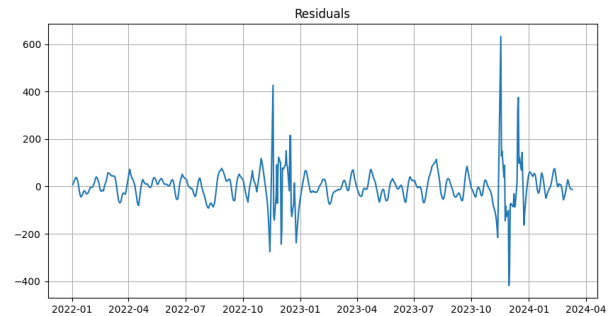


Figure 15: Residuals for polynomial interpolation.

The calendar effect refers to variations in time series data caused by the structure of the calendar [7]. This includes the distribution of weekdays, weekends, and holidays throughout the year. In the context of sales data, the calendar effect can significantly influence sales patterns. Sales might spike on weekends and drop on holidays when stores

are closed or other factors that might influence consumer behavior changes. Modeling the calendar effect is essential for several reasons. Firstly, ignoring the calendar effect can lead to inaccurate forecasts. By accounting for holidays and variations in the number of working days, we can improve the precision of the model's sales predictions. Lastly, understanding the impact of the calendar effect helps businesses, including Maki, plan for changes in sales more effectively, aligning them with periods of higher consumer activity [31] [32].

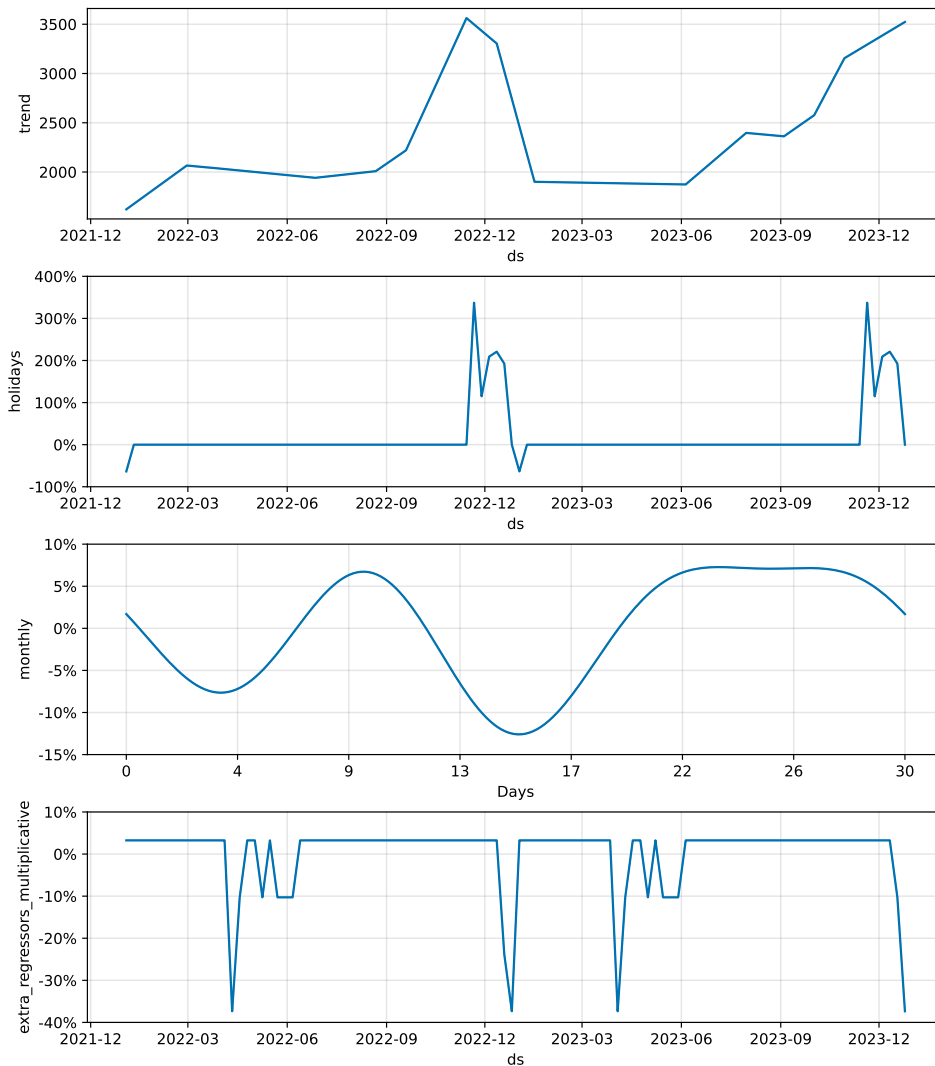


Figure 16: Final model components.

The calendar effect is pronounced in daily sales data, as sales fluctuate significantly between weekdays, weekends, and holidays. Accurate modeling requires identifying and adjusting for these variations. There might also be a general tendency for holidays to land on a particular weekday, and it is thus crucial to model this for improving accuracy. The calendar effect is further smoothed out in monthly data but can still influence sales trends, particularly in months with major holidays, like Easter. Adjustments are necessary to account for the varying number of weekends and holidays each month. The calendar effect is less volatile but still significant for weekly aggregated data. We are always guaranteed for a week to consist of the days Monday to Sunday. However, weeks with

holidays might show reduced sales, necessitating adjustments in weekly sales forecasts. To effectively model the calendar effect in sales data from stores in Denmark, we leverage the Python package `holidays` to retrieve the list of public holidays specific to Denmark. This choice is pertinent because the dataset in question comprises sales data from Danish stores as described when introducing the dataset in Section 2.1. Danish holidays significantly influence consumer behavior and sales patterns, making it essential to account for these holidays. A closing law governs Danish retail, which is a set of regulations that dictates the closing hours of stores. Given this law, we assume a seven-day working week. However, holidays still impact the number of operational days in a week.

Therefore, we adjust the number of working days by subtracting holidays from the week. This is done by incorporating the adjusted working days count as a regressor in the Prophet model. This allows the model to automatically adjust the impact of the calendar effect in its predictions by finding the optimal β parameter in Section 3.4.

Monthly seasonality is the last component added to our final Prophet model, along with the calendar effect. With ACF and PACF in Section 2.4, we highlight the necessity of incorporating a monthly seasonality. Like the yearly seasonality, the monthly seasonality is estimated with a Fourier series. Taylor et al. [6] suggest a Fourier order of 4 when estimating the monthly component. The final model then consists of trend, yearly seasonality, monthly seasonality, holidays, and calendar effect observed on Figure 16, and the weeks included as holiday regressors consist of the first week and weeks 47 – 51.

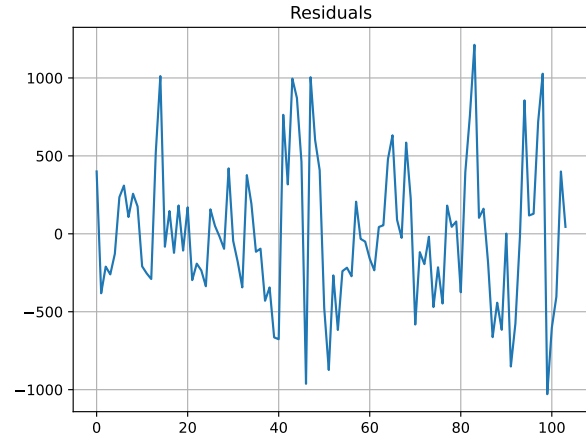
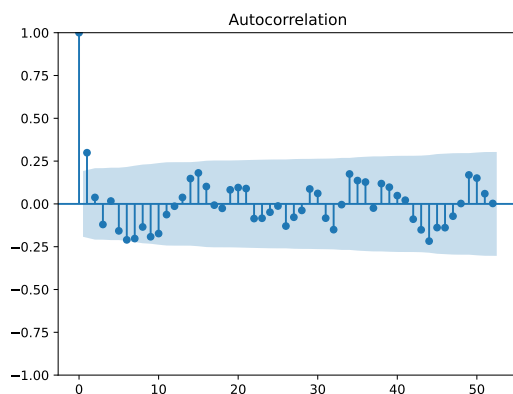
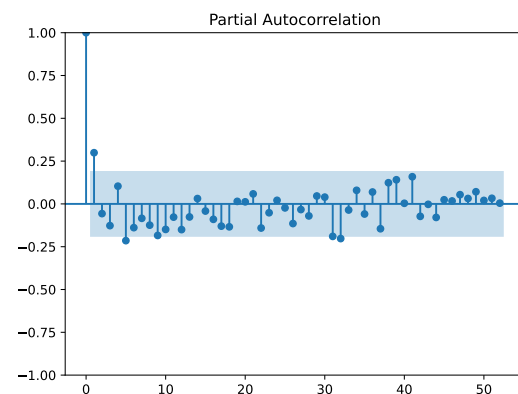


Figure 17: Final model residuals.



(a) Final model ACF plot.



(b) Model residuals.

Figure 18: Final model PACF plot.

Residuals are plotted on Figure 17 and appear to be i.i.d. We plot the ACF and PACF to see if the final model composition can fully fit the data complexity. Some significant lags

can still be observed on Figure 18, hinting at a sequential correlation between lags. To ensure the conclusion, the residuals are tested for sequential dependency using the Ljung-Box test, observed on Figure 19. The null hypothesis is that no significant autocorrelation exists for a specific lag [33].

A p-value below 0.05 for all lags signifies the rejection of the null hypothesis, meaning there is a sequential correlation, and the residuals are not yet i.i.d. The final model composition, therefore, fails to address the temporal dynamics of the provided sales series.

5.2 Generating test data

A theme in this project is the sparsity of data. We highlight the issue of using cross-validation on only 2022-23 in Section 2.6. Using only two years in cross-validation results in only one completed cycle of seasonality when validating model hyperparameters. It is necessary to include some of the 2024 data to accommodate this. Including the 2024 data in the validation process eliminates it for further testing because we will fabricate a look-ahead bias. This produces a deadlock. We cannot use the 2024 data for training and validation, however, not including it will provide us with flawed results due to decomposition requirements. Figure 18 and Figure 19, shows our model do not completely capture the seasonalities, and other approaches needs to be applied. Models such as the ARIMA family mentioned in Section 1.2 and other differencing techniques can be applied to ensure complete stationarity. However, we do not view these as an immediate solution. Our goal is to parameterize the trend, such that its trajectories and growths can be resonated, and therefore Prophet is the obvious choice.

Our immediate solution to the deadlock and lack of data is simulation. Residuals are shown to be normally distributed on several occasions. Our solution is then to approximate this distribution and simulate future residuals by sampling from this distribution. We have not completely modeled our way to i.i.d. residuals, however, sampling from the approximate distribution will yield i.i.d. data, which is representative of the residuals Maki would typically see in their day-to-day business. Simulating data will further allow labeled changepoints in a test setup, as we can inject them as needed.

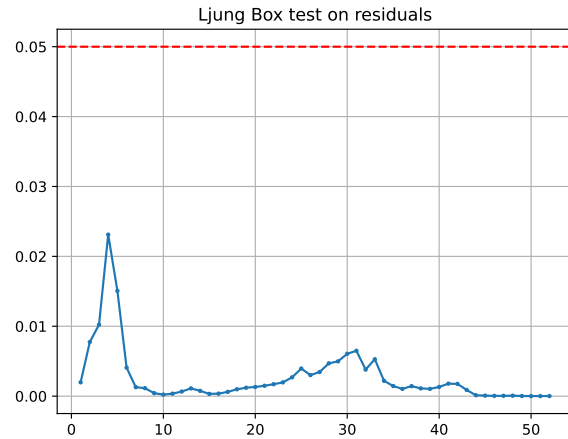


Figure 19: Residual Ljung-Box test.

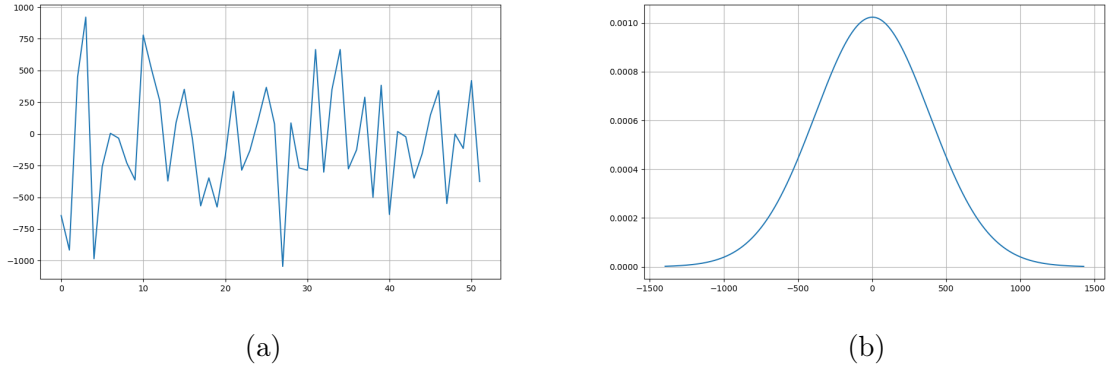


Figure 20: Figure 20a shows generated residuals. Figure 20b shows the probability density function of the t-distribution from which the residuals were sampled.

The manner in which we generate the data leverages Prophet’s predictive capabilities and sampling from an approximated distribution. We obtain residuals by fitting Prophet on all available data and producing in-sample forecasts. We then approximate parameters of the Normal-Gamma distribution based on these residuals. The Normal-Gamma is a conjugate prior to a normal distribution with unknown mean and variance. Because of their conjugate relationship, the posterior distribution parameters can be directly computed from the prior parameters and the data [34].

Given a Normal-Gamma prior and observed data $x = (x_1, x_2, \dots, x_n)$, the posterior parameters are updated as follows [34]. The posterior mean μ_n is given by:

$$\mu_n = \frac{\kappa_0 \mu_0 + n \bar{x}}{\kappa_0 + n}$$

where μ_0 is the prior mean, κ_0 is the prior precision, $\bar{x}$ is the sample mean, and n is the sample size. The posterior precision κ_n is given by:

$$\kappa_n = \kappa_0 + n$$

The posterior shape α_n is given by:

$$\alpha_n = \alpha_0 + \frac{n}{2}$$

where α_0 is the prior shape parameter. The posterior rate β_n is given by:

$$\beta_n = \beta_0 + 0.5 \sum_{i=1}^n (x_i - \bar{x})^2 + \frac{\kappa_0 n (\bar{x} - \mu_0)^2}{2(\kappa_0 + n)}$$

where β_0 is the prior rate parameter. We further transform the Normal-Gamma distribution to a t-distribution for practical reasons. The t-distribution closely resembles the normal distribution but with heavier tails. Computation of the t-distribution parameters, given the parameters of the Normal-Gamma distribution, are as following [34]:

Location : μ

Degrees of Freedom : 2α

$$\text{Scale} : \sqrt{\frac{\beta(\kappa + 1)}{\alpha\kappa}}$$

An example of the probability density function for the t-distribution is provided in Figure 20b.

A significant challenge in changepoint detection is verifying changepoints. Zhou et al. [25], utilizes domain expert opinions to validate their research. However, they note that many changepoint applications can be too complex to rely on expert assessment of changepoints, and relying solely on data labeled by humans may not always be appropriate. We obtain any desired length and count of time series by sampling from the residual distribution as showcased in Section 5.2. This setup allows us to test our proposed method of a warning system in a controlled environment, where we can produce any number of samples we see fit to test the robustness and sensitivity of our approaches. What constitutes a changepoint is not immediate and, more so, something we typically cannot validate with certainty. We solve this by imputing changepoints into generated data, characterized by either mean shifts in the underlying data generative process or a time-dependent growth, thereby exerting ground truths into the data. This simplifies the evaluation of the changepoint algorithms. The experimental setups are explained in detail in the following chapter.

6 Experiments

After addressing the theoretical foundations and key challenges, Figure 21 provides an overview of this study's experiment pipeline, illustrating all processes and their interactions.

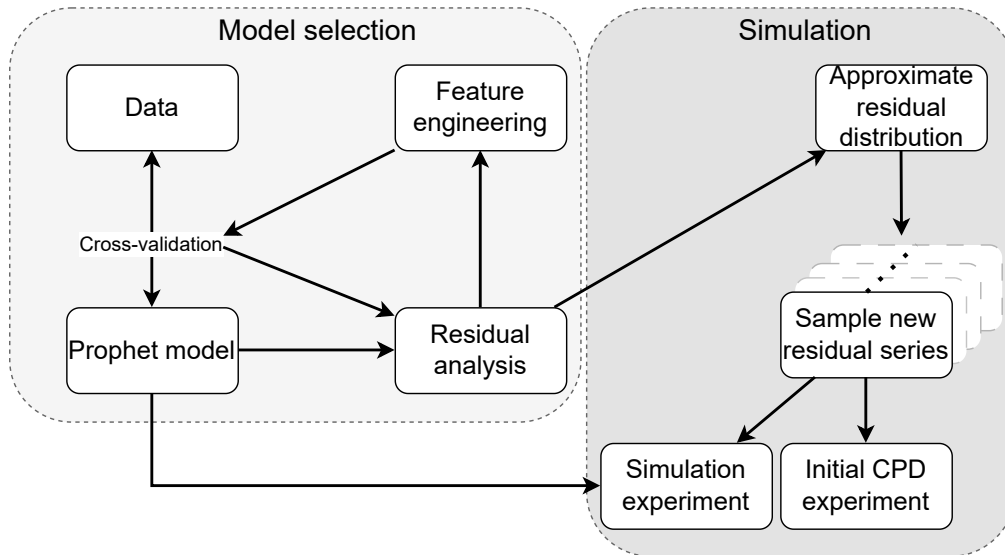


Figure 21: Data and model pipeline.

Figure 21 shows the model selection process encompassing the evaluation of relevant hyperparameters, choices regarding training length, and analysis of residuals. This process is iterative and involves extensive feature engineering to enhance model performance.

Feature engineering is, therefore, an integral component of model selection and is included in the overall methodology.

The experiments and simulations rely on the residual analysis conducted during the model selection phase. The distribution of residuals is approximated and subsequently utilized in the experiments. These experiments are defined in the subsequent sections.

6.1 Prophet configuration

As a prerequisite to testing changepoint algorithms, we evaluate and identify the optimal hyperparameters for the Prophet model. Detailed in Figure 21, we incorporate Prophet for changepoint detection analysis in two ways. We obtain residuals from differencing Prophet forecasts and observed data. We sample new residual series using a distribution with parameters approximated from the residuals. They are particularly useful for tuning the BOCPD and PELT algorithms, as they are representative of the actual data. Prophet is also used to generate missing test data when attempting to mimic a real-world scenario. Prophet forecasts 52 weeks ahead, supplying 52 weeks of generated data to which we add residuals sampled from the aforementioned distribution. Hence, a properly fitted Prophet model ultimately carries significant weight in our ability to pinpoint changepoints and showcase meaningful results.

Cross-validation is done by utilizing the full dataset. Fitting the model on all 2022-2023 data provides an adequate amount to properly capture the seasonal component. Subsequently, the model forecasts four weeks ahead to which errors are computed. As a final step, the training windows shifts two weeks and the process repeats until the end of test set is reached, thus obtaining errors over 14 observations in total.

On top of adding dates of salient holidays relevant for consumer spending habits, we add the regressor presented in Section 5.1. Beyond the holiday, seasonality and regressor components detailed here, there are multiple other hyperparameters of the Prophet model, which we tune, summarized in Table 4. We decide on the best hyperparameters by the ones that yield the minimum mean absolute percentage error (MAPE) $\frac{1}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100$.

Table 4: Overview of Hyperparameters

Hyperparameter	Description	Candidates
Seasonality mode	Mode for seasonality component	Additive, Multiplicative
Holiday mode	Mode for holiday component	Additive, Multiplicative
Changepoint prior scale	Scale for changepoint flexibility	0.01, 0.02, 0.05, 0.1
Seasonality prior scale	Scale for seasonality component prior	3, 5, 8, 10
Holidays prior scale	Scale for holidays component prior	3, 5, 8, 10

6.2 Changepoint detection experiments

To evaluate the performance of the changepoint detection algorithms, we introduce two parameters: the magnitude and the injected changepoint index. The magnitude is systematically adjusted to understand its impact on the detection capabilities. The parameters are observed in Table 5.

Table 5: Parameter descriptions and values.

Parameter	Description	Value
Magnitude	The manually imputed changepoint	100, 200, . . . , 3000
Changepoint index	The index for the imputed magnitude	25

We introduce only one changepoint into the generated series for simplicity and to maintain control over the experimental conditions. The magnitude is a constant value added at a specific time step in the generated series and to all subsequent data points, resembling a change in the series’ mean. The magnitude represents the abrupt change in the series; in this study, it is defined as a sudden increase in sales. The magnitude of this change is important; if it is not substantial enough, the changepoint algorithms might fail to detect it as a changepoint, or they might identify the change only after a significant delay. This delay in detection can be critical in time-sensitive applications, affecting decision-making processes. However, it is important to note that the algorithm’s inability to detect a change with an insignificant magnitude is not inherently problematic; it reflects the algorithms’ sensitivity to changes.

The metric measuring the number of steps refers to the number of steps or timestamps required for the changepoint detection algorithm to identify the change post-occurrence. Timely changepoint detection is important given the project’s domain setting, where Maki seeks to identify trend changes within a maximum of 12 weeks after their occurrence. With this experiment, we are mainly interested in two types of measures. We want to know how well the algorithms can correctly identify the changepoint and how many steps past the changepoint the algorithm needs to identify a change that has occurred. We, therefore, structure the experiment partly as a classification problem and partly as a quantitative measure. Various magnitudes are introduced to analyze the changepoint detection algorithms’ reactivity to change. This analysis helps us understand how different magnitudes affect the detection speed and accuracy of the algorithm.

Additionally, we examine the true positive rate, false negative rate, and distribution of steps across magnitudes to evaluate the algorithm’s robustness. We define a true positive rate as the cases where the algorithm returns only the correct index of the changepoint and no other. In contrast, the false negative rate measures the instances where the algorithm fails to detect a changepoint that is present, no matter the size of the mean shift. The distribution of steps at a given magnitude level provides insight into the consistency of the algorithm’s performance across multiple experiments. Ideally, one would verify that the magnitude does not exceed an upper limit relevant to the domain setting. This ensures that the introduced changepoints are realistic and applicable to real-world scenarios.

To ensure the robustness of our findings, this experiment is repeated by generating 10,000 sample time series with a length of a year, that is, 52 data points, and performing the

described analysis on each sample. This large-scale experimentation comprehensively explains the algorithms' performances under various conditions. In Section 7.2, we will discuss the performance of PELT and BOCPD in detecting changepoints, highlighting their strengths and potential areas for improvement based on the observed metrics and analysis.

PELT configuration

Before executing the experiment, we employ the Python package `Optuna` for hyperparameter tuning to determine the optimal penalty for the PELT cost function, introduced in Equation (7). The optimal penalty value is defined as the lowest penalty that does not result in the detection of a changepoint. The penalty values are normalized with min-max normalization. If a penalty value results in a falsely placed changepoint, its value is set to $1 + 0.1 * N$, where N is the number of falsely placed changepoints. We do this to encourage a richer search space for `optuna`. This approach ensures that the algorithm is not overly sensitive to noise and only identifies significant changes within the data. PELT also includes additional parameters such as `min_size`, which defines the minimum size of each segment, a `jump` parameter that allows for accelerated calculations by skipping certain data points, and a `cost` parameter that enables the selection between two cost functions: the L1 norm and the L2 norm. For our study, we set these additional parameters to fixed values to impose the least constraint on the algorithm. Specifically, we set the `min_size` and the `jump` parameters to 1. These settings minimize constraints on the algorithm, allowing it to function with maximum flexibility. We select the L2 norm as the cost function.

BOCPD configuration

The first step of BOCPD requires defining a prior distribution for the data's parameters. Moreover, we must choose a changepoint prior to the Hazard function. The library used for implementing BOCPD, named `Changepoint` [35], offers a variety of priors for different purposes, namely the Normal-Gamma, Beta-Bernoulli, and Poisson-Gamma. The Beta-Bernoulli prior is appropriate for binary data, the Poisson-Gamma prior for count data, and the Normal-Gamma prior for continuous normally distributed data with unknown mean and variance. Given the nature of our data, the Normal-Gamma is the only appropriate choice for the prior.

The changepoint prior in the hazard function is typically chosen based on domain knowledge and prior expectation of changepoint frequencies. The `Changepoint` library, however, only includes a geometric distribution resulting in the Hazard function becoming $H(\tau) = \frac{1}{\lambda}$. λ is chosen based on the expected time between changepoints, which we set to 52, simply because it is the full length of the generated series, and we expect a single changepoint.

6.3 Real world changepoint detection experiment

The real-world experiment aims to emulate a realistic application scenario for Maki. The experiment closely mirrors the algorithmic experiments conducted for PELT and BOCPD but includes several distinct differences to reflect practical conditions better. We split the experiment into two distinct parts.

In the first part, the methodology closely follows the earlier described experiment in Section 6.2 but differs significantly in how the test data is generated. For this part, the Prophet model generates 52 weeks of test data. To enhance the realism of the test set, the residuals generated from the Prophet model on the training data, as detailed in Section 5.2, are added to the test data. This step introduces randomness, providing a more realistic representation of real-world fluctuations. We view this first part purely as a classification problem, where the changepoint is the positive class, and any other point is the negative class. We allow the algorithm to place a single changepoint, and then evaluate its classification metrics based on that. This gives an imbalanced dataset, as the negative class will outweigh the positive class at 51 to 1. This implies true negative rate has a more semantic meaning rather than useful information due to the heavy class imbalance.

The second part of the experiment adopts a different approach. Instead of treating the experiment as a classification problem, where an abrupt change was introduced into the series, this part involves inserting a relatively small, continuous change. We increment the series with a pre-defined magnitude over a number of time steps, resulting in a linear increase in trend. The magnitudes used in this part of the experiment are as follows: 50, 100, 150, 300, and 500. This experiment aims to answer the following questions regarding the detection capabilities of the changepoint algorithms: how many steps are required to detect a changepoint? Where is the changepoint detected relative to when the magnitude was first introduced?

The evaluation of results in this experiment uses an additional metric. The Hausdorff measure is employed to assess the robustness of the changepoint detection methods. This metric is particularly powerful as it relaxes the strict binary constraint of correct or incorrect guesses. Instead, it measures the largest distance between the true changepoint and the detected changepoint, allowing for a more nuanced evaluation of the algorithm's performance. The Hausdorff measure captures the proximity of the detection to the actual changepoint, thus providing a more flexible and comprehensive assessment of the detection methods' robustness. The Hausdorff measure is defined as

$$\text{HAUSDORFF}(T^*, \hat{T}) := \max \left\{ \max_{\hat{t} \in \hat{T}} \min_{t^* \in T^*} |\hat{t} - t^*|, \max_{t^* \in T^*} \min_{\hat{t} \in \hat{T}} |t^* - \hat{t}| \right\},$$

and describes the worst error made by the algorithm. T^* denotes the set of true changepoints, and $\hat{T}$ denotes the set of the changepoints predicted by the algorithm. If this metric is equal to zero, both changepoint sets are equal. It is large when a changepoint from either T^* or $\hat{T}$ is far from every changepoint of $\hat{T}$ or T^* , respectively. Over-segmentation, as well as under-segmentation, is penalized. We split the Hausdorff into two groups to highlight the worst errors for the positive cases instead of the general cases. We define these groups as **Hausdorff distance for true positives** and **Hausdorff distance for all**, respectively. The Hausdorff distance for true positive cases highlights the furthest distance between the index where a magnitude is introduced and the furthest changepoint predicted after that index. The Hausdorff distance for all cases describes the furthest distance between the index where the magnitude is introduced and any other predicted changepoint. This means it also calculates the distance for changepoints detected before the index where a magnitude is introduced.

We also introduce the metric **average steps**, which quantifies the average number of

steps required for the algorithm to detect a change around the index where the magnitude is introduced. This metric inherently refers to the number of steps following the introduction of the magnitude, not preceding it. The metric **standard deviation steps** measures the standard deviation of the number of steps required for the algorithm to detect the changepoint at the index where the magnitude is introduced. Together with **average steps**, the **standard deviation steps** is important for assessing whether the detection times meet the domain specification of a reaction time between 8 to 12 weeks. We include an additional classification metric, the F1 score. The F1 score is included as it represents the harmonic mean of precision and recall, providing a balanced measure of both metrics in a single value. The F1 score is calculated as $\frac{2tp}{2tp+fp+fn}$ and ranges from zero to one, where a value of one indicates perfect precision and recall, and a value of zero indicates that both precision and recall are zero.

By incorporating these modifications, the real-world experiment aims to provide a more accurate and practical evaluation of the changepoint detection methods, reflecting the complexities and nuances of real-world data and application scenarios.

7 Results & discussion

This section presents the findings of the study based on the experiments described in Section 6.3. First we cover the optimal hyperparameters in model selection, a key result for the following experiments in changepoint detection.

7.1 Prophet model hyperparameters

Table 6 presents the hyperparameter tuning results for the final Prophet model, comparing configurations with and without including the working days regressor. The tuned hyperparameters include the seasonality mode, holidays mode, changepoint prior scale, seasonality prior scale, and holidays prior scale. The table also provides the MAPE for both configurations.

Table 6: Comparison of hyperparameters with and without regressor

Hyperparameter	With Regressor	No Regressor
Seasonality mode	Additive	Additive
Holidays mode	Multiplicative	Multiplicative
Changepoint prior scale	0.05	0.02
Seasonality prior scale	3	5
Holidays prior scale	3	10
MAPE	14.58	32.47

A significant improvement emerges with the inclusion of the regressor, reducing MAPE from 32.47% to 14.58%. We conclude that adding information about the number of opening days benefits the model. The scale for the changepoint, seasonality, and holiday prior can hinder their ability to react to changes. Larger scales increase flexibility, and in

the absence of a regressor, one can postulate that the increased scales signal an attempt to capture the missing regressor information with extreme values for the seasonality and holiday components. Larger prior scales enable the exposure to overfitting, potentially explaining the higher MAPE in this case. In conclusion, the reported values in the second column of Table 6 are used onward with the working days regressor.

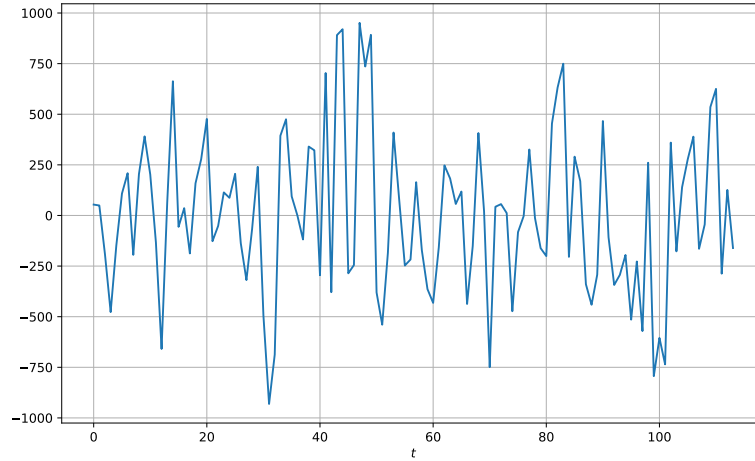


Figure 22: Residuals from in-sample forecast.

We conclude in Section 5.1 that the model specification produced normally distributed residuals yet were still autocorrelated. The success criterion of this experiment ultimately becomes to achieve constant variance in the residuals (heteroscedasticity).

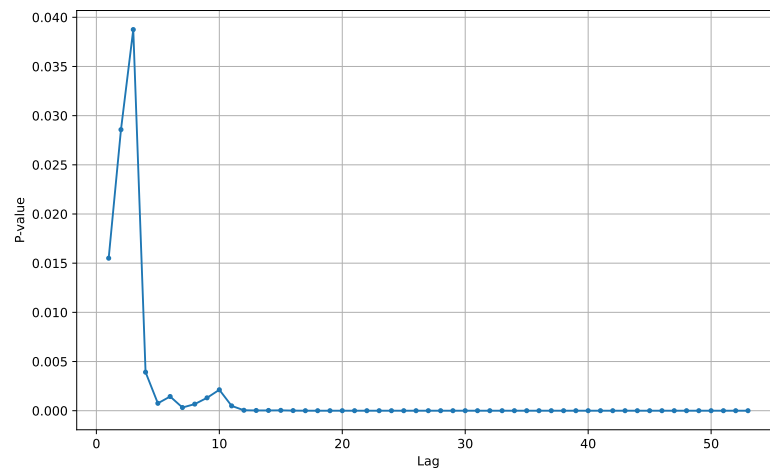


Figure 23: P-values for lag 1-53 in a Ljung-Box test. Null-hypotheses is that there is no autocorrelation.

We test the i.i.d. assumption with a Ljung-Box autocorrelation test, examining the presence of significant autocorrelation. Given a significance level $\alpha = 0.05$, we reject the null hypothesis for all lags in Figure 23, indicating a clear presence of autocorrelation in

the residual series. We have not been able to model the sales data adequately. While a good Prophet model is instrumental for our ability to detect changepoints, adding extra regressors in the form of exogenous variables, which potentially improves the model, is infeasible under the current setup. We end this chapter by noting that the imperfect model could harm our ability to detect changepoints later.

7.2 Changepoint detection results

In this section, we outline the results from the discussion in Section 6.2, providing an analysis of the performance of PELT and BOCPD under various conditions.

Pelt Results

The hyperparameter tuning resulted in the identification of the optimal penalty value for the PELT algorithm. This optimal penalty value was found to be 3.059.130.

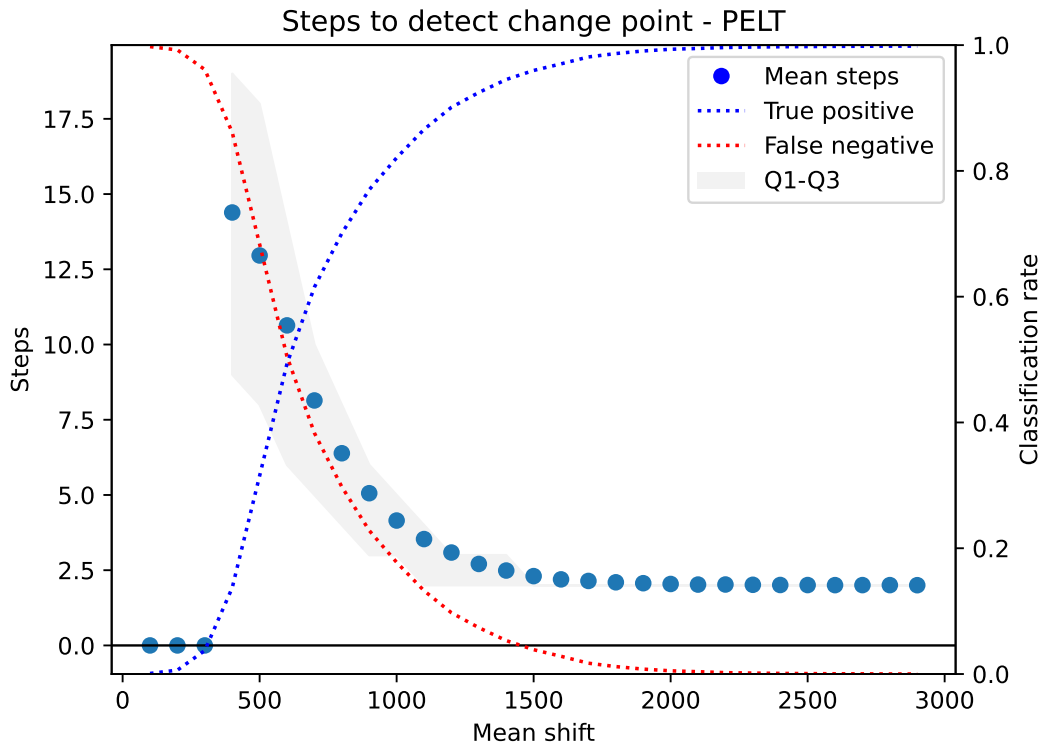


Figure 24: PELT changepoint detection result

Figure 24 illustrates the relationship between the magnitude of the manually imputed changepoint and various performance metrics of the PELT algorithm. On the x-axis, the figure displays the magnitude values, while the left y-axis represents the number of steps required for PELT to identify the changepoint. The gray area in the figure denotes the upper and lower quantiles of the steps, presenting the consistency of the algorithm. The right y-axis depicts the true positive rates and the false negative rates. The figure reveals that for magnitude values of 100, 200, and 300, PELT cannot detect these changes as changepoints. As discussed in the experiment section, this is not necessarily a negative

outcome; it simply reflects the algorithm's sensitivity. These smaller magnitudes might be too insignificant to warrant detection as changepoints.

The first changepoint detection occurs at a magnitude of 300. However, the data indicates that this detection is inconsistent, as evidenced by the relatively high false negative rate and the corresponding low true positive rate. When PELT does identify a changepoint at this magnitude, the detection occurs, on average, approximately 15 steps (or weeks) after the changepoint has occurred. The wide gray area representing the upper and lower quantiles highlights the variability in the number of detection steps. When the magnitude reaches 700, there is a noticeable improvement in the detection performance. The true positive rate surpasses the false negative rate, and the detection time falls within the acceptable range of a maximum of 12 weeks. However, even at this magnitude, the true positive rate might not be sufficient for practical applications, such as those required by Maki. A higher magnitude of change is necessary for consistent and reliable detection in real-world scenarios. This increased magnitude improves the true positive rate and reduces the variance, leading to more consistent detection performance. Nonetheless, PELT also requires a larger abrupt change in sales which might not be desirable. These results suggest that while PELT can detect changepoints, its performance is highly dependent on the magnitude of change. For practical applications, especially in contexts requiring timely and reliable detection, careful consideration must be given to the sensitivity of PELT.

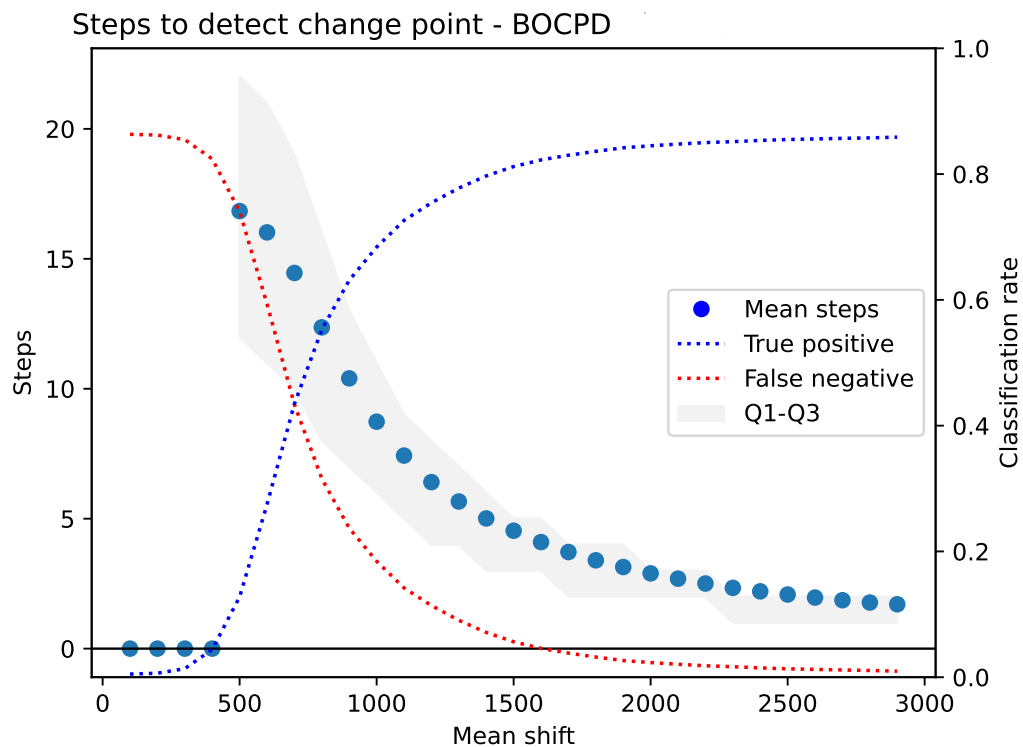


Figure 25: BOCPD changepoint detection result

BOCPD Results

We conducted the same experiment using the BOCPD algorithm, and the results are shown in Figure 25. A notable difference between BOCPD and PELT is immediately apparent: a shift in the y-axis indicates that BOCPD is generally slower at detecting changepoints. Additionally, BOCPD requires a larger magnitude change before it detects changepoints. We could choose a lower value for λ in the hazard function, prompting more frequent detection. However, this adjustment must be approached cautiously, as it potentially comes with a trade-off with the false positive rate.

In terms of true positive rates, PELT outperforms BOCPD. BOCPD reaches an upper limit of around 0.85, even with magnitude changes greater than 2000. In contrast, PELT stabilizes at 1 when magnitude changes exceed 2000, demonstrating significantly better performance.

Overall, both algorithms achieve their goal of detecting changes in the mean of the data generative distribution. However, PELT generally requires fewer steps and is more precise. Moving forward, we will investigate their performance in a real-world application context.

7.3 Real world experiment results

Lastly, we present the results from the experiments designed to evaluate the algorithms in a real-world application context. The analysis is divided into two sections. First, we discuss the results from the experiment assessing the algorithms' performance on abrupt changes. Second, we examine the algorithms' performance when a continuous change is introduced to the test data.

Table 7: Performance metrics of the BOCPD algorithm at different magnitudes

Magnitude	TPR	FPR	FNR	TNR	F1 Score
1500	0.75	0.005	0.25	0.995	0.75
1000	0.56	0.009	0.44	0.991	0.56
500	0.13	0.006	0.87	0.994	0.178

Table 8: Performance metrics of the PELT algorithm at different magnitudes

Magnitude	TPR	FPR	FNR	TNR	F1 Score
1500	0.92	0.002	0.08	0.998	0.92
1000	0.79	0.004	0.21	0.996	0.79
500	0.28	0.007	0.72	0.993	0.34

Table 7 and Table 8 show the results from the first experiment for BOCPD and PELT, respectively. We have selected a subset of magnitudes based on the patterns observed in Figure 24 and Figure 25. In the first experiment, the abrupt changes are designed to mimic sudden shifts in the trend that might occur in real-world scenarios, such as those

observed for item 0-2 in Figure 4. The true negative rate remains consistently high due to the experimental design, which involves only a single positive point to detect midst numerous negative points. Consequently, the abundance of negative points leads to a high true negative rate. Conversely, the false positive rate is also influenced by this design, albeit in the opposite manner. The scarcity of positive points results in a relatively low false positive rate.

Comparing the F1 scores of the two algorithms, it is evident that PELT consistently outperforms BOCPD across all magnitudes. PELT's higher F1 scores indicate that this algorithm is more precise and sensitive in detecting changepoints. The fact that PELT maintains higher F1 scores across varying magnitudes suggests that it offers a more reliable and accurate detection mechanism for abrupt changes in real-world data scenarios.

Table 9: Comparison of BOCPD and PELT metrics

Metric	Method	Values
Hausdorff distance for true positives	BOCPD	13, 10, 7, 4, 3
	PELT	23, 11, 8, 6, 4
Hausdorff distance for all	BOCPD	23, 23, 23, 23, 23
	PELT	23, 11, 8, 6, 4
Average steps	BOCPD	20.75, 14.1, 11.1, 7.3, 5.6
	PELT	17.53, 11.2, 8.87, 5.9, 4.37
Standard deviation steps	BOCPD	3.9, 1.7, 1.3, 0.66, 0.66
	PELT	3.92, 1.70, 1.31, 0.65, 0.66

Table 9 presents the results from the experiment in which a continuous change is introduced to the test data. The magnitudes selected are 50, 100, 150, 300, and 500, respectively. The **Values** column in Table 9 represents the values for a given metric corresponding to the same sequence of magnitude values. For instance, considering the Hausdorff metric for true positives, the values are 13, 10, 7, 4, and 3 for the BOCPD method. These values indicate that, for the BOCPD method, the worst distance between the introduction of the magnitude and when a changepoint is detected is 13 (equivalent to 13 weeks in this study) when the magnitude is 50. This implies that at the index by the 13th week, the series has shifted by an amount equal to 13 times the magnitude of 50. Similarly, when the magnitude is 100, it takes a distance of 10 weeks for the method to detect a change. This pattern highlights that BOCPD can detect lesser changes, given enough time. In the worst case, the cumulative magnitude shift for 50 is detected at 650, whereas the worst cumulative case for magnitude 500 is detected at 1500. The proportional reduction in the number of steps needed to detect changes as the magnitude increases suggests that BOCPD is more effective at identifying changes as they gradually accumulate over time rather than sudden abrupt shifts.

In contrast, examining the same metric for the PELT method reveals a different performance profile. PELT shows significantly poorer performance on small continuous changes

but performs better with larger abrupt changes. This indicates that while PELT is highly effective at detecting substantial, sudden shifts in the data, it struggles to identify more gradual, continuous changes. Overall, these results demonstrate the differing strengths and sensitivities of the two algorithms.

When examining the **Hausdorff distance for all metric**, we observe that the BOCPD method consistently registers a value of 23. This indicates that BOCPD detects a changepoint in the test data at the very beginning of the series. BOCPD appears susceptible to its online formulation. In contrast, as an offline changepoint detection algorithm, PELT analyzes all available data before determining the optimal changepoint. This analysis results in stable values for both the **Hausdorff distance for true positives** and **Hausdorff distance for all metrics**, reflecting the method's holistic approach to data analysis.

Analyzing the values for the **average steps** and **standard deviation steps** metrics reveals that when a magnitude of 150 is introduced, the detection is just within the specified range of 8 to 12 weeks, with an average detection time of 11.1 steps and a standard deviation of 1.3 steps. These values decrease for higher magnitudes of 300 and 500, indicating that the models are both faster and more consistent in identifying the changepoint at the specified index.

The experiments highlight distinct performance characteristics of the BOCPD and PELT methods, each exhibiting their strengths and limitations. The BOCPD method demonstrated remarkable sensitivity to small changes over time in the series, consistently detecting changepoints early in the data. This sensitivity makes BOCPD particularly suitable for applications requiring real-time detection and fast response to evolving data patterns.

The PELT method benefits from its ability to analyze the entirety of the available data before determining the optimal changepoints. This comprehensive approach results in stable and consistent performance across various metrics, making PELT well-suited for scenarios where a holistic understanding of the data is crucial. Nonetheless, PELT's reliance on the complete dataset can limit its responsiveness to real-time changes.

8 Conclusion and future work

Maki inquires on an early warning system that provides timely and accurate insights into consumer trends and market dynamics. We tackle this problem by utilizing an established forecast method in Meta’s Prophet and the changepoint algorithms BOCPD and PELT, which, in unison, solve this problem. Specifically, we propose a pipeline in which we obtain residuals from Prophet forecasts and the observed data and subsequently detect statistical changes in the underlying residual distribution using BOCPD and PELT.

We identify and provide answers to the modeling of seasonal patterns in the data. Following a hyperparameter tuning, we report a mean absolute percentage error of 14.58%, although the main conclusion is drawn from the non-i.i.d. residuals, revealing insufficient data modeling. Changepoint detection results show that BOCPD excels in adaptability and quick detection, and PELT offers robustness and stability. Neither method can be deemed superior to the other. A hybrid approach, leveraging the real-time sensitivity of BOCPD and the comprehensive analysis of PELT, may offer significant advantages, providing both quick and robust detections. Ultimately, Maki’s requirements and situational needs should determine the choice between BOCPD and PELT or a combination thereof. Thus, the results of the experiments in Section 6.3 serve as a foundation for Maki to determine the desired balance between reactivity and sensitivity, ensuring that the chosen model aligns with their strategic objectives and operational constraints.

The first task for our future work is eliminating the residuals’ autocorrelation. We deem the sparse data insufficient, and more data could aid in better feature engineering and understanding of the temporal dependencies. Currently, exogenous variables are not included in this project. However, this is aligned with the provided data being trend stationary, and the annual sales are nearly identical. Most socioeconomic variables would seem needless for explaining no changes. However, some of the seasonal volatility could potentially have been accommodated by more exogenous information.

The project only provides experiments on the total sales time series. When given more data, a natural extension is to test BOCPDs and PELTs on item-level and brand-level time series. Maki demands early detection at this granularity to make it relevant as a consultant for adjusting purchasing quantities. Thus, it would ultimately serve as a proof of concept as to whether our setup is applicable in a real-world context.

Our approach comes with issues, as our changepoint detection algorithms completely depend on the goodness-of-fit of our prophet model. The prophet model usage was two-fold. Firstly, we want to identify workable components to parameterize the trend. Secondly, we obtain a residual series to apply our changepoint algorithms by subtracting the model’s prediction. However, this method will be flawed if we want to continually ingrain new information into the model as new data becomes available. Instead, we could adopt a similar approach, like X-13ARIMA-SEATS, to deseasonalize the series and apply the changepoints algorithms directly.

9 Bibliography

- [1] OpenAI. *ChatGPT: Optimizing Language Models for Dialogue*.
<https://openai.com/blog/chatgpt>. Accessed: 2024-05-23. 2022.
- [2] Friedman, Nat. *Introducing GitHub Copilot: your AI pair programmer*.
<https://copilot.github.com/>. Accessed: 2024-05-23. 2021.
- [3] R.J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*.
Available at: OTexts.com/fpp3. Accessed on April 15, 2024.
OTexts: Melbourne, Australia, 2021.
- [4] George EP Box et al. *Time series analysis: forecasting and control*.
John Wiley & Sons, 2015.
- [5] Maki. URL: <https://www.maki.dk/about/> (visited on 04/28/2024).
- [6] Sean J. Taylor and Benjamin Letham. “Forecasting at Scale”. In: *PeerJ Preprints* 5 (2017). <https://doi.org/10.7287/peerj.preprints.3190v2>, e3190v2.
- [7] F.X Diebold. *Econometric Data Science: A Predictive Modeling Approach*.
Department of Economics, University of Pennsylvania, 2019.
- [8] Manfred Mudelsee. “Trend analysis of climate time series: A review of methods”.
In: *Earth-Science Reviews* 190 (2019).
<https://doi.org/10.1016/j.earscirev.2018.12.005>, pp. 310–322.
- [9] Yasaman Ensafi et al. “Time-series forecasting of seasonal items sales using machine learning – A comparative analysis”.
In: *International Journal of Information Management Data Insights* 2 (2022).
<https://doi.org/10.1016/j.ijime.2022.100058>, p. 100058.
- [10] Australian Bureau of Statistics. “Interpreting Time Series Data”.
In: *Western Australian Statistical Indicators* 1367.5 (2002). March.
URL: <https://www.abs.gov.au>.
- [11] Mette Kramer Pedersen and Charlotte Franck Fæste. “Seasonal adjustment of Danish financial time series using the X-12-ARIMA procedure”. In: 44 (2006).
URL: <https://hdl.handle.net/10419/82366>.
- [12] Columbia University Mailman School of Public Health. *Box-Jenkins Methodology*.
<https://www.publichealth.columbia.edu/research/population-health-methods/box-jenkins-methodology>.
- [13] Estela Bee Dagum and Silvia Bianconcini.
Seasonal Adjustment Methods and Real Time Trend-Cycle Estimation.
Statistics for Social and Behavioral Sciences. Springer, 2016.
ISBN: 978-3-319-31820-2. DOI: 10.1007/978-3-319-31822-6.
- [14] Kwangwon Lee.
Quarterly National Accounts Manual - Chapter 7: International Accounts.
International Monetary Fund. 2017.
- [15] European Central Bank. *Seasonal Adjustment*.
<https://www.ecb.europa.eu/pub/pdf/other/statseasonaladjustmenten.pdf>.
- [16] Bryan Lim and Stefan Zohren.
“Time-series forecasting with deep learning: a survey”.
In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 379 (2194 2021). DOI: 10.1098/rsta.2020.0209.
URL: <https://royalsocietypublishing.org/doi/10.1098/rsta.2020.0209>.

- [17] Google Research. *A Decoder-Only Foundation Model for Time-Series Forecasting*. <https://research.google/blog/a-decoder-only-foundation-model-for-time-series-forecasting/>. 2024.
- [18] Kashif Rasul et al. “Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting”. In: *arXiv* abs/2310.08278 (2023). URL: <https://arxiv.org/pdf/2310.08278>.
- [19] Bineet Kumar Jha and Shilpa Pande. “Time Series Forecasting Model for Supermarket Sales using FB-Prophet”. In: *Proceedings of the Fifth International Conference on Computing Methodologies and Communication (ICCMC 2021)*. IEEE Xplore, 2021. DOI: 10.1109/ICCMC51019.2021.9418033.
- [20] Charles Truong, Laurent Oudre, and Nicolas Vayatis. “Selective review of offline change point detection methods”. In: *Signal Processing* 167 (2020). DOI: 10.1016/j.sigpro.2019.07.024.
- [21] Samaneh Aminikhanghahi and Diane J. Cook. “A survey of methods for time series change point detection”. In: *Knowledge and Information Systems* 51 (2017), pp. 339–367. DOI: 10.1007/s10115-016-0987-z. URL: <https://link.springer.com/article/10.1007/s10115-016-0987-z>.
- [22] Manfred Mudelsee. “Trend analysis of climate time series: A review of methods”. In: *Earth-Science Reviews* 190 (2019), pp. 310–322. DOI: 10.1016/j.earscirev.2018.12.005. URL: <https://www.sciencedirect.com/science/article/pii/S0012825218303726>.
- [23] Reza Habibi. “Bayesian Online Change Point Detection in Finance”. In: *Financial Internet Quarterly* 17.4 (2021), pp. 27–33. DOI: 10.2478/fiqf-2021-0025. URL: <https://doi.org/10.2478/fiqf-2021-0025>.
- [24] Han Zhang and Qian Zhou. *Change Point Detection methods applied to Financial Time Series*. Research Report. Directed by Dr. Pablo Roldan, June 2021.
- [25] Kaiguang Zhao et al. “Detecting change-point, trend, and seasonality in satellite time series data to track abrupt changes and nonlinear dynamics: A Bayesian ensemble algorithm”. In: *Remote Sensing of Environment* 232 (2019), p. 111181. DOI: <https://doi.org/10.1016/j.rse.2019.04.034>.
- [26] Gerrit J. J. van den Burg and Christopher K. I. Williams. “An Evaluation of Change Point Detection Algorithms”. In: *arXiv* arXiv:2003.06222v3 (Feb. 2022). URL: <https://arxiv.org/abs/2003.06222v3>.
- [27] Ryan Prescott Adams and David J. C. MacKay. *Bayesian Online Changepoint Detection*. 2007. arXiv: 0710.3742 [stat.ML].
- [28] Gregory Gundersen. *Bayesian Online Changepoint Detection*. Accessed: 2024-05-20. 2019. URL: <https://gregorygundersen.com/blog/2019/08/13/bocd/>.
- [29] Paul Bretz. *Changepoint Detection: A Survey and Comparative Study*. https://math.ou.edu/~pbretz/Changepoint_Detection.pdf. Accessed: 2024-05-23. 2024.

- [30] Rebecca Killick, Paul Fearnhead, and Idris A Eckley.
“Optimal detection of changepoints with a linear computational cost”.
In: *Journal of the American Statistical Association* 107.500 (2012), pp. 1590–1598.
- [31] Peter Reinhard Hansen, Asger Lunde, and James M Nason.
“Testing the significance of calendar effects”.
In: *Federal Reserve Bank of Atlanta Working Paper* 2005-02 (2005).
- [32] Laurens Swinkels and Pim Van Vliet. “An anatomy of calendar effects”.
In: *Journal of Asset Management* 13.4 (2012), pp. 271–286.
- [33] Pennsylvania State University. *Lesson 3: Independence and Dependence. STAT 510 - Applied Time Series Analysis*. Accessed: 2024-05-23. 2024.
URL: <https://online.stat.psu.edu/stat510/lesson/3/3.2>.
- [34] J. Staphorsius and S. Abrams.
Inference and Decision Making with Multiple Parameters. Accessed: 2024-05-23. 2020. URL: <https://statswithr.github.io/book/inference-and-decision-making-with-multiple-parameters.html>.
- [35] Promised AI. *changepoint: Bayesian Changepoint Detection*.
<https://github.com/promised-ai/changepoint/tree/master>.
Accessed: 2024-05-23. 2024.

A Appendix - Brand and item overview

We provide a table with item prefixes. This is meant for the reader to connect items with brands if such mapping is necessary.

Table 10: List of Brands, Item Prefixes, and Number of Items

Brand	Items prefix	Number of items
DR	B-*	7
Fantasma	D-*	2
Fortnite	F-*	1
Hape	H-*	12
Krea	J-*	4
Magformers	K-*	4
Mattel Games	M-*	1
Monster Jam	N-*	1
Paw Patrol	O-*	3
Schleich	P-*	49
Tech Deck	Q-*	4
Vildkatten	R-*	3
Vtech	S-*	15
Wheely Bug	T-*	4

B Appendix - Optimal Partitioning

Let $\mathcal{T}_s = \{\tau : 0 = \tau_0 < \tau_1 < \dots < \tau_m < \tau_{m+1} = s\}$ be the set of possible vectors of changepoints and set $F(0) = -\beta$, then

$$\begin{aligned}
F(s) &= \min_{\tau \in \mathcal{T}_s} \left\{ \sum_{i=1}^{m+1} [C(y_{(\tau_{i-1}+1):\tau_i}) + \beta] \right\}, \\
&= \min_t \left\{ \min_{\tau \in \mathcal{T}_t} \left[\sum_{i=1}^m (C(y_{(\tau_{i-1}+1):\tau_i}) + \beta) + C(y_{(t+1):n}) + \beta \right] \right\}, \\
&= \min_t \{F(t) + C(y_{(t+1):n}) + \beta\}.
\end{aligned}$$

This provides a recursion which gives the minimal cost for data $y_{1:s}$ in terms of the minimal cost for data $y_{1:t}$ for $t < s$. Note this function aims to minimize $\sum_{i=1}^{m+1} [C(y_{(\tau_{i-1}+1):\tau_i}) + \beta]$ equivalent to Equation (7) where $f(m) = m$.